From Gradients to Curvature and Matrix Geometry: A Survey of Optimization Methods for Deep Neural Networks

Faris Nasser and Salah Moussa

***Abstract*—Optimizers determine how a neural network turns a gradient into a parameter update. A scalar learning rate gives gradient descent; momentum incorporates history; Adam and related algorithms rescale individual coordinates; Newton and natural-gradient methods use curvature or information geometry; and contemporary methods such as Shampoo, SOAP, and Muon exploit the matrix structure of neural-network weights. Despite their shared objective of minimizing training loss, these algorithms use fundamentally different notions of distance, memory, and computational work. This survey develops a mathematically connected account of these methods for readers familiar with calculus, linear algebra, probability, and introductory machine learning. We first derive the gradient, Hessian, stochastic-gradient model, smoothness bounds, conditioning, and a variable-metric interpretation of descent. We then review classical and adaptive first-order optimizers; Newton, quasi-Newton, Hessian-free, natural-gradient, and structured curvature approximations; and modern memory-efficient and matrix-aware alternatives. Throughout, we distinguish true Hessian information from the Fisher matrix, gradient statistics, and spectral-norm geometry. A common comparison framework separates convergence guarantees from empirical performance and accounting identities from implementation-dependent resource claims. The survey concludes that optimization methods should be compared by the information they use, the geometry they impose, and their end-to-end cost rather than by age or iteration count alone.**

***Index Terms*—Non-convex optimization, stochastic gradient descent, Adam, Hessian, Newton method, quasi-Newton, natural gradient, K-FAC, Shampoo, Sophia, Muon, preconditioning, neural networks.**

I. Introduction

TRAINING a neural network is an optimization problem, but the textbook instruction "follow the negative gradient" leaves several crucial decisions unanswered. How accurately should the gradient be estimated? Should the update depend on previous gradients, and by how much? Is one learning rate sufficient for all parameters? Could curvature distinguish a steep narrow valley from a gently sloping plateau? When a parameter is a matrix, should the optimizer preserve its row-and-column structure instead of flattening it into a long vector? These questions motivate a large family of optimizers whose mathematical principles are related but whose costs and guarantees can differ substantially [6], [8], [9].

The design problem is particularly demanding in deep learning. Even a network whose individual layers are linear is usually non-convex in the collection of its weight matrices. Training data introduce stochasticity through minibatches; different parameter directions may have very different scales; and large models make optimizer state and matrix factorizations materially expensive. Backpropagation supplies gradients efficiently [7], but it does not prescribe how those gradients ought to be transformed. Standard practice has therefore evolved from stochastic gradient descent (SGD) and momentum toward adaptive coordinate-wise methods, structured preconditioning, and matrix-aware updates [16], [31], [68], [74].

A. Existing Surveys and Scope

The literature is sufficiently broad that a useful survey requires a principled boundary. Bottou, Curtis, and Nocedal [9] provide a foundational treatment of large-scale machine-learning optimization; Nocedal and Wright [6] develop classical unconstrained optimization, including Newton, quasi-Newton, and trust-region methods. Work on deep-learning landscapes emphasizes critical points and curvature [43]–[45], while recent optimizer papers focus on practical trade-offs in memory, geometry, and large-model training [32], [72], [73]. These works have different scopes. Rather than survey every algorithm ever proposed, we trace how different constructions of the update direction address particular limitations of an ordinary gradient step.

We focus on *single-objective, unconstrained empirical-risk minimization*. We include deterministic and stochastic methods, methods using Hessian information directly or approximately, and matrix-aware algorithms that are *not* Hessian approximations. Learning-rate schedules, normalization, and distributed training enter only when they change the comparison. Multi-objective and constrained descent are outside scope. We do not claim to reproduce the empirical results of cited papers; numerical performance is attributed explicitly to its original study, and illustration slots are reserved for reproducible figures.

F. Nasser and S. Moussa are with the College of Engineering, American University of Sharjah, Sharjah, United Arab Emirates.

Prepared for MLR 570, Advanced Machine Learning, Fall 2026. This is a literature survey; figure slots indicate planned illustrations, not experimental results.

B. Research Questions

Two connected questions organize the main text:

- **RQ1 (gradient transformation).** How do stochastic, momentum, adaptive, and memory-efficient first-order methods transform gradient information, and what limitations do these transformations address?
- **RQ2 (curvature and geometry).** How do second-order, curvature-aware, and matrix-aware methods exploit information beyond scalar gradient scaling, and what convergence, memory, compute, and stability trade-offs follow?

We deliberately avoid asking "Which optimizer is best?" without fixing the model, loss, compute budget, batch size, and evaluation protocol. A method can have excellent performance per update yet require more time per update; its convergence theorem may hold for a convex model while a reported speedup comes from non-convex transformer experiments.

C. Contributions and Organization

The survey makes four contributions as a *synthesis*, not a new algorithm. First, it derives the mathematical tools—stochastic gradients, smoothness, curvature, and conditioning—before introducing update rules. Second, it uses an explicit variable-metric model to connect ordinary GD, diagonal adaptation, Newton, and natural gradient, while explaining where that analogy fails for Muon. Third, it separates *what is estimated* (Hessian, Fisher, gradient moments, or a matrix norm) from *how the estimate is used* (inversion, clipping, momentum, or orthogonalization). Fourth, it provides a consistent comparison of optimizer state, per-step arithmetic, guarantees, and empirical evidence.

Section II supplies the mathematical foundation. Section III reviews first-order families and memory-efficient extensions, answering RQ1. Sections IV and V develop classical curvature methods and modern structured or matrix-aware updates, answering RQ2. Section VI critically compares the families, Section VII identifies open problems, and Section VIII concludes. Figure 1 is reserved for a conceptual map of this progression.

II. Mathematical Preliminaries

A. From Data to a Non-Convex Objective

For samples $\{(x_i, y_i)\}_{i=1}^n$, model f_θ, and per-example loss ℓ, empirical risk is

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i), \qquad \theta \in \mathbb{R}^d. \tag{1}$$

The parameter vector θ collects weights and biases, even though software usually stores each layer's weights as matrices or tensors. For squared loss and a linear model $f_\theta(x) = x^\top\theta$, L is a convex quadratic. For a multilayer network, f_θ composes parameterized linear maps with activation functions; products of weights already suffice to make L non-convex. Consequently, a descent algorithm need not find a *global* minimizer, and the curvature may change sign across the landscape [35], [43].

The objective should not be confused with a metric of test performance. Training loss describes optimization on the sample; generalization concerns new data. These can differ even when the optimizer finds a low-loss point [26], [39], [40]. This distinction is important when interpreting studies that compare final validation accuracy rather than optimization progress.

B. Gradient, Hessian, and Local Taylor Models

The gradient is $g(\theta) = \nabla L(\theta)$ with coordinate $g_j = \partial L/\partial\theta_j$. It is the vector of first derivatives. For a small displacement Δ, the first-order Taylor model is

$$L(\theta + \Delta) \approx L(\theta) + g^\top\Delta. \tag{2}$$

The Cauchy–Schwarz inequality $g^\top\Delta \geq -\|g\|_2\|\Delta\|_2$ implies that, among displacements with a fixed Euclidean length, $-g$ gives the largest *first-order* decrease. This is the geometric meaning of "steepest descent"; it does not mean a finite step always decreases the true nonlinear loss.

The Hessian $H(\theta) = \nabla^2 L(\theta) \in \mathbb{R}^{d\times d}$ contains second derivatives $H_{jk} = \partial^2 L/(\partial\theta_j\partial\theta_k)$. For twice differentiable L, its local quadratic model is

$$L(\theta + \Delta) \approx L(\theta) + g^\top\Delta + \tfrac{1}{2}\Delta^\top H\Delta. \tag{3}$$

The quadratic term answers a question the gradient cannot: *how rapidly will the slope change as we move?* For an eigenpair $Hv = \lambda v$ with $\|v\|_2 = 1$, displacement $\Delta = av$ contributes $\frac{1}{2}\lambda a^2$. Positive λ means locally upward curvature; negative λ means a direction of decreasing curvature; $\lambda \approx 0$ means a flat local model. At a stationary point $g = 0$, negative curvature rules out a local minimum, while $H \succeq 0$ is necessary but not sufficient for a local minimum unless stronger conditions hold [6], [43].

C. Smoothness, the Descent Lemma, and Safe Steps

A differentiable loss is β-smooth on a region if

$$\|g(x) - g(y)\|_2 \leq \beta\|x - y\|_2. \tag{4}$$

For twice differentiable functions, this is implied locally by $\|H(\theta)\|_2 \leq \beta$ on that region. Integrating the gradient along the line from x to $x + \Delta$ yields the *descent lemma*:

$$L(x + \Delta) \leq L(x) + g(x)^\top\Delta + \frac{\beta}{2}\|\Delta\|_2^2. \tag{5}$$

Substitute the GD update $\Delta = -\eta g(x)$:

$$L(x - \eta g) \leq L(x) - \eta\|g\|^2 + \frac{\beta\eta^2}{2}\|g\|^2 \tag{6}$$

$$= L(x) - \eta(1 - \tfrac{\beta\eta}{2})\|g\|^2. \tag{7}$$

Thus $0 < \eta < 2/\beta$ guarantees nonincrease *under these smoothness conditions*; choosing $\eta \leq 1/\beta$ gives a convenient quantitative bound. This derivation illustrates why learning rate and curvature cannot be discussed independently [5], [9], [47].

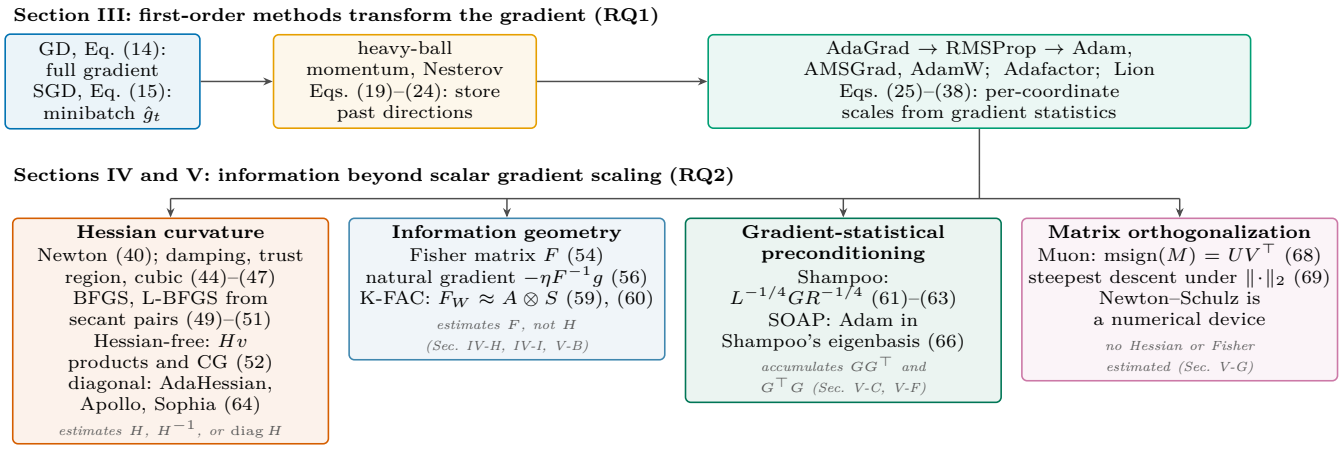


Fig. 1: Organization of the survey. The branches separate Hessian curvature, information geometry, gradient-statistical preconditioning, and matrix orthogonalization; their similar algebra does not make them the same approximation.

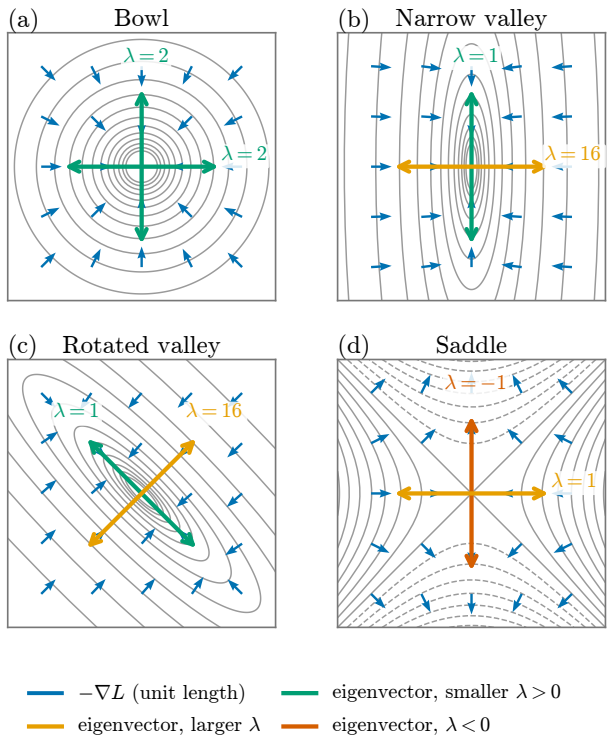


Fig. 2: Loss geometry in two dimensions. Gradients indicate local slope; Hessian eigenvectors indicate directions of principal curvature. The bowl and saddle are local models, not claims about global network landscapes.

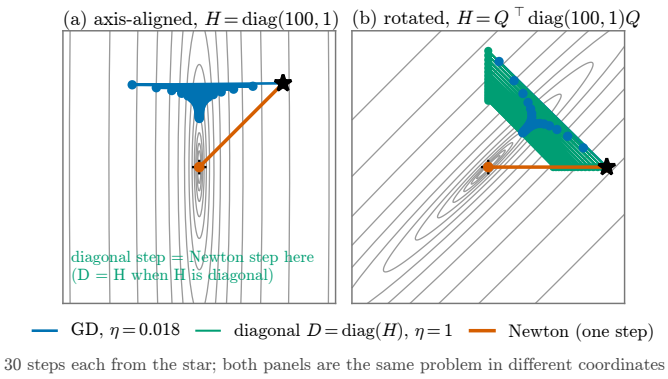


Fig. 3: Why conditioning and coordinate choice matter. The Hessian eigenvalues are unchanged by rotation, but diagonal rescaling is tied to the chosen coordinates; full-matrix transformations can follow rotated principal axes.

D. Curvature, Conditioning, and Rotated Valleys

Consider the quadratic

$$L(x, y) = \tfrac{1}{2}(100x^2 + y^2), \qquad H = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}. \tag{8}$$

Its level sets are elongated ellipses: movement in the x direction is one hundred times more curved than in y. If $\eta = 0.01$, GD multiplies x by $1 - 100\eta = 0$ and y by $1 - \eta = 0.99$ per iteration. The large-curvature direction is solved immediately in this specially selected example, while the low-curvature direction progresses slowly. The condition number $\kappa = \lambda_{\max}/\lambda_{\min} = 100$ captures this anisotropy; on general strongly convex quadratics, optimal fixed-step GD still converges at a rate governed by κ [5], [6].

Now rotate the coordinates by an orthogonal matrix Q and replace H with $Q^\top \text{diag}(100, 1)Q$. The eigenvalues are unchanged, but the principal axes no longer coincide with the stored parameter coordinates. Diagonal scaling can change how far we move along each coordinate, yet cannot generally remove the off-diagonal coupling induced by the rotation. Full-matrix and structured methods aim to represent such interactions, albeit at additional cost. Figure 3 should show both level sets and trajectories.

E. Local Linear Stability and Non-Convex Stationarity

Near a stationary point $\theta^\star$ where $g(\theta^\star) = 0$, a first-order expansion of the gradient gives $g(\theta^\star + e) \approx H^\star e$. The GD error update is therefore

$$e_{t+1} \approx (I - \eta H^\star)e_t. \tag{9}$$

TABLE I: The information used by representative optimizer families. "Curvature" is not a synonym for every matrix-valued update.

Family	Constructed quantity	Geometric interpretation
SGD / momentum	Gradient, past gradients	Euclidean update with history
Adam / Adafactor	Squared-gradient statistics	Coordinate-dependent scaling
Newton / BFGS	Hessian or secant approximation	Local quadratic curvature
Natural gradient / K-FAC	Fisher-information geometry	Distributional displacement
Shampoo / SOAP	Structured gradient statistics	Tensor-coordinate preconditioning
Muon	Orthogonalized momentum matrix	Spectral-norm steepest direction

In each Hessian eigenvector direction the multiplier is $1-\eta\lambda$. For $0<\eta\lambda<2$, the local component contracts; if $\lambda<0$, it expands without sign reversal; if $\eta\lambda>2$, it expands while alternating sign. The famous $2/\lambda_{\max}$ threshold belongs to this *local quadratic analysis*; it is not a universal global step-size rule for arbitrary networks [42].

For non-convex objectives, a first-order stationary point satisfies $\|g(\theta)\|\le\epsilon$. An approximate second-order stationary point additionally satisfies $\lambda_{\min}(H(\theta))\ge-\gamma$. Gradient-based guarantees usually concern a small gradient norm, not global optimality. Saddle-escape results require separate assumptions and sometimes randomized perturbations [44]–[46].

F. Stochastic Gradients: What Minibatches Change

With a minibatch B_t sampled appropriately from the data, write

$$\hat{g}_t=\frac{1}{|B_t|}\sum_{i\in B_t}\nabla L_i(\theta_t)=g_t+\xi_t,\qquad \mathbb{E}[\xi_t\mid\theta_t]=0. \quad (10)$$

Unbiasedness is an assumption about the sampling protocol; it is not automatic in every data pipeline. With independent samples of per-example gradient variance bounded by σ^2, the minibatch estimator has variance at most $\sigma^2/|B_t|$. It is cheaper than a full gradient but perturbs the update. Smaller batches can provide more frequent parameter updates per data pass, while larger batches change arithmetic intensity and parallel efficiency; hardware and tuning complicate the comparison [1], [2], [8], [10], [30].

To see the effect of noise in a convergence bound, substitute $\Delta=-\eta\hat{g}_t$ into Eq. (5), take conditional expectation, and use $\mathbb{E}\|\hat{g}_t\|^2=\|g_t\|^2+\mathbb{E}\|\xi_t\|^2$:

$$\mathbb{E}_t[L_{t+1}]\le L_t-\eta(1-\tfrac{\beta\eta}{2})\|g_t\|^2+\frac{\beta\eta^2\sigma^2}{2|B_t|}. \quad (11)$$

Noise adds a variance term, producing a trade-off between faster descent and the stochastic error floor. Non-convex SGD theory formalizes this with assumptions on smoothness, variance, and sampling [9], [11].

G. A Unifying Variable-Metric Model—and Its Limits

For a positive-definite matrix M_t, consider the local subproblem

$$\Delta_t=\arg\min_{\Delta}\left[g_t^\top\Delta+\frac{1}{2\eta}\Delta^\top M_t\Delta\right]. \quad (12)$$

Differentiating with respect to Δ and setting to zero gives $g_t+M_t\Delta/\eta=0$, hence

$$\boxed{\Delta_t=-\eta M_t^{-1}g_t.} \quad (13)$$

This is a convenient lens for multiple methods. Euclidean GD uses $M_t=I$; diagonal adaptation chooses a diagonal, history-dependent M_t; a positive-definite Hessian yields a Newton-like step; a Fisher matrix yields natural gradient. But this is a *conceptual* relationship, not an assertion that Adam's gradient moments estimate a Hessian. Nor does it exactly describe Muon, whose spectral-norm geometry is generally not a quadratic metric. These distinctions motivate the taxonomy in Table I [13], [61], [63], [75].

H. What Counts as Cost or Convergence?

A parameter vector with d entries requires $O(d)$ numbers to store, but the complete Hessian has d^2 entries. For an $m\times n$ layer, a full layer-wise metric over mn entries requires m^2n^2 numbers, while row and column factors require m^2+n^2; a factored second-moment vector requires approximately $m+n$. These are storage orders, not end-to-end byte estimates. Master copies of parameters, gradients, momentum, sharding, and mixed precision all affect actual peak memory [31], [68], [76], [77].

Iteration complexity is equally easy to misread. A result of $O(1/T)$ for function error in convex optimization cannot be ranked against $O(1/\sqrt{T})$ for expected squared gradient norm in non-convex stochastic optimization. Different quantities, noise models, and accuracy criteria are involved. The paper compares theorems only after identifying their target quantity and assumptions [5], [9], [11].

III. First-Order Optimization: From Gradients to Adaptivity (RQ1)

A. Gradient Descent and Stochastic Gradient Descent

Gradient descent uses the full gradient to update every parameter:

$$\theta_{t+1}=\theta_t-\eta_t\nabla L(\theta_t). \quad (14)$$

The step has two interpretable parts. The direction $-\nabla L$ comes from the linear model in Eq. (2); the positive scalar η_t determines how far the update trusts that model. If a derivative is $g=8$ and $\eta=0.1$, the parameter decreases by 0.8; if $g=-3$, the same rule *increases* the parameter by 0.3. Subtracting a negative gradient is an ordinary signed subtraction, not an instruction to make every parameter smaller. The descent lemma formalizes why choosing η too large can overturn the local prediction.

SGD replaces the full gradient with Eq. (10),

$$\theta_{t+1}=\theta_t-\eta_t\hat{g}_t. \quad (15)$$

It was developed in the stochastic-approximation tradition of Robbins and Monro [1] and the broader numerical-statistics literature [2]. In modern machine learning, minibatching amortizes data access and exploits parallel devices. A full-data step may be more accurate, yet it can require far more work than a minibatch step. Consequently "iterations to a given loss" and "time to a given loss" should be reported separately [8], [9].

A basic deterministic non-convex result can be derived without advanced machinery. Suppose L is bounded below by L_* and β-smooth, and choose $\eta \le 1/\beta$. Sum Eq. (7) over $t = 0, \dots, T-1$ to obtain

$$\frac{\eta}{2} \sum_{t=0}^{T-1} \|g_t\|^2 \le L(\theta_0) - L(\theta_T) \tag{16}$$

$$\le L(\theta_0) - L_*. \tag{17}$$

Therefore

$$\boxed{\min_{0 \le t < T} \|g_t\|^2 \le \frac{2[L(\theta_0) - L_*]}{\eta T}.} \tag{18}$$

For a constant η, obtaining a gradient norm no larger than ϵ takes $O(\epsilon^{-2})$ iterations in this worst-case upper bound. The conclusion is *first-order stationarity*, not recovery of the globally best network. Stochastic versions must additionally control the variance term in Eq. (11) [11], [24].

B. Momentum: Why Store Past Directions?

The difficulty of simple GD on a narrow valley is not only that steps may be small: alternating gradients across the steep direction create zigzags, while the shallow direction progresses slowly. Polyak's heavy-ball method introduces a velocity:

$$v_t = \mu v_{t-1} + g_t, \tag{19}$$

$$\theta_{t+1} = \theta_t - \eta v_t, \qquad 0 \le \mu < 1. \tag{20}$$

Unrolling the recurrence yields $v_t = g_t + \mu g_{t-1} + \mu^2 g_{t-2} + \cdots$ (taking $v_{-1} = 0$). Thus a gradient direction that persists over time accumulates weight, while alternating signs partly cancel. Unlike a generic average of historical parameters, momentum averages *directions*. This insight predates deep learning [3] and remains relevant to training sensitive networks [12].

Momentum can also overshoot. For the one-dimensional quadratic $L(\theta) = \lambda\theta^2/2$ and full-batch gradients $g_t = \lambda\theta_t$, eliminate the velocity using $\theta_t - \theta_{t-1} = -\eta v_{t-1}$:

$$\theta_{t+1} = (1 + \mu - \eta\lambda)\theta_t - \mu\theta_{t-1}. \tag{21}$$

A solution of the form $\theta_t = r^t$ gives the characteristic polynomial

$$r^2 - (1 + \mu - \eta\lambda)r + \mu = 0. \tag{22}$$

Its roots determine whether the trajectory contracts, oscillates, or grows. This shows mathematically why the momentum coefficient μ and learning rate η must be tuned *together*. For a positive scalar λ and $0 \le \mu < 1$, local stability requires $0 < \eta\lambda < 2(1 + \mu)$; this restricted

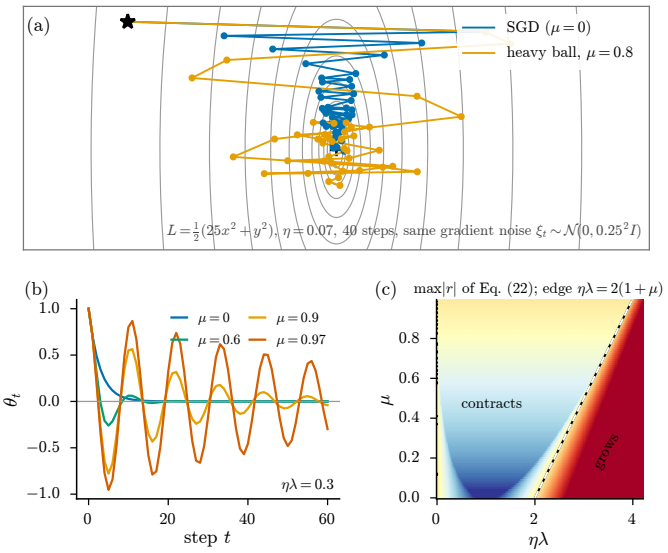


Fig. 4: Momentum integrates directions over time. Across a narrow valley, alternating components may cancel, but excessive momentum can create long-lived oscillations or instability.

quadratic statement should not be generalized to arbitrary changing neural-network Hessians. Classical acceleration results assume convexity or stronger structure [3], [5].

C. Nesterov Acceleration: A Gradient at the Look-Ahead Point

Nesterov acceleration modifies the relationship between momentum and gradient evaluation. One commonly used deep-learning form is

$$y_t = \theta_t + \mu(\theta_t - \theta_{t-1}), \tag{23}$$

$$\theta_{t+1} = y_t - \eta\nabla L(y_t). \tag{24}$$

First extrapolate to y_t using past movement, then compute the gradient there. This differs from heavy-ball momentum, which uses the current gradient and an accumulated velocity. The exact accelerated algorithm and its guarantees depend on the coefficient schedule and objective class: for smooth convex objectives, Nesterov's classical analysis yields an $O(1/T^2)$ function-value rate under appropriate settings, versus the usual $O(1/T)$ for basic convex GD [4], [5]. That accelerated rate does *not* automatically hold for the general non-convex neural-network case.

A useful interpretation is that the look-ahead point exposes some of the change in slope that the velocity would encounter after moving. Nonetheless, extrapolation is based on gradients and past iterates, not explicit Hessian computation. We classify Nesterov acceleration as a first-order method. In practice, its benefits and tuning depend on initialization, batch regime, and the interaction with network parameterization [12], [35], [36].

D. AdaGrad: Learning Rates for Different Coordinates

A scalar learning rate assumes the same scale should be used for every parameter. Sparse or heterogeneous fea-

tures violate that intuition. AdaGrad accumulates squared coordinate gradients,

$$s_{t,j} = s_{t-1,j} + \hat{g}_{t,j}^2, \tag{25}$$

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\eta}{\sqrt{s_{t,j}} + \varepsilon}\,\hat{g}_{t,j}. \tag{26}$$

A coordinate with frequent large gradients builds a large denominator and receives smaller future updates, whereas a rarely activated coordinate may retain a larger effective step. Consider two features whose cumulative squared gradients are 100 and 1. Ignoring ε, their effective learning rates are $\eta/10$ and η respectively. This simple example reveals the origin of *coordinate-wise* adaptation without claiming that the accumulated squares are Hessian entries.

AdaGrad arises naturally from online optimization and time-varying proximal geometry; its original analysis concerns regret and stochastic/online convex settings [13]. The trade-off is visible directly in Eq. (25): $s_{t,j}$ never decreases, so effective learning rates can become very small in long training runs. A fixed cumulative history cannot rapidly forget early gradients when the loss landscape changes. AdaGrad is therefore important as both an original method and the motivation for exponential moving averages in later adaptive optimizers.

E. RMSProp and Exponential Gradient Statistics

RMSProp replaces AdaGrad's cumulative sum with an exponential moving average,

$$s_t = \rho s_{t-1} + (1-\rho)\hat{g}_t^{\odot 2}, \qquad \theta_{t+1} = \theta_t - \eta\frac{\hat{g}_t}{\sqrt{s_t} + \varepsilon}, \tag{27}$$

where powers, roots, and divisions act elementwise. Because older squared gradients are discounted, the scale estimate can adapt when the training trajectory enters a different region. RMSProp was popularized in Hinton's lecture material [14]; AdaDelta proposed a related running-statistics design [15]. Theoretical analysis of their unconstrained, non-convex variants needs explicit assumptions and cannot be inferred from the appealing heuristic alone.

The smoothing factor ρ introduces a new timescale. If ρ is near one, the denominator changes slowly and averages over a long history; if it is small, it reacts quickly but inherits more minibatch noise. Therefore a method that "automatically adjusts learning rates" still contains parameters that determine how that adjustment behaves. This subtlety reappears in Adam's β_2 and in the stability analysis of adaptive methods [18], [19].

F. Adam: Momentum Combined with Adaptive Scaling

Adam combines a first-moment estimate m_t and a second *raw* moment estimate v_t of the stochastic gradient [16]:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\hat{g}_t, \tag{28}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\hat{g}_t^{\odot 2}. \tag{29}$$

With zero initialization, the exponential averages initially underestimate the corresponding stationary moments. For

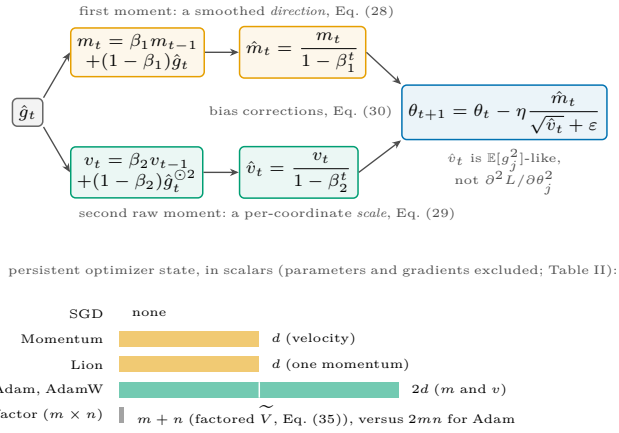


Fig. 5: Adam builds two different gradient statistics. The first moment estimates direction; the second moment controls coordinate-wise scaling. Neither equals the Hessian.

a constant deterministic gradient g and $m_0 = 0$, recurrence solving gives $m_t = (1-\beta_1^t)g$. Dividing by $1-\beta_1^t$ removes this initialization bias in that example. Adam applies analogous corrections:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \tag{30}$$

These equations are the core of Adam; they also explain its two parameter-shaped state arrays. The second moment is $\mathbb{E}[g_j^2]$-like, not a Hessian diagonal. Even when the optimizer rescales gradients in a way that resembles curvature correction on some objectives, it does not follow that $\hat{v}_t \approx \partial^2 L/\partial\theta_j^2$.

A small numerical example clarifies the update. If a coordinate's corrected first moment is $\hat{m} = 0.3$ and corrected second moment is $\hat{v} = 0.09$, its update is approximately $-\eta$ when ε is negligible. If $\hat{m} = 0.3$ but $\hat{v} = 0.36$, it is approximately -0.5η. This relative scaling is the main practical idea; exact values depend on the history and smoothing choices. Because the update is not merely $-\eta g_t$, Adam is not generally invariant to a change of parameter coordinates, despite useful invariances to certain gradient rescalings discussed in its original paper [16].

G. Adam's Convergence Caveat and AMSGrad

Popularity is not a convergence theorem. Reddi, Kale, and Kumar [18] constructed a simple online convex setting in which Adam's original exponential weighting can fail to converge, and proposed AMSGrad to retain a nondecreasing running maximum of the second-moment scale. The essential modification is

$$\bar{v}_t = \max(\bar{v}_{t-1}, v_t), \qquad \Delta_t \propto -\frac{m_t}{\sqrt{\bar{v}_t} + \varepsilon}. \tag{31}$$

Here the maximum is coordinatewise. The lesson is not that Adam always diverges or AMSGrad always wins empirically; rather, smooth heuristic adaptations can have

subtle stability conditions. Subsequent convergence analyses of Adam-type methods explicitly control effective step-size variation, moment schedules, or bounded gradients [19], [20], [25]. RAdam examined variance effects during the early adaptive-learning-rate phase [21]; AdaBelief and AdaBound proposed further changes to adaptive scaling [22], [23]. These are meaningful research threads, but their theoretical statements are not directly comparable unless their settings match.

H. AdamW: Why Regularization and Weight Decay Are Different

For a loss $L(\theta)$ with quadratic penalty $(\lambda/2)\|\theta\|^2$, the penalized gradient is $g + \lambda\theta$. With ordinary SGD,

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta g_t. \tag{32}$$

Thus adding an L_2 penalty and multiplying weights by $1-\eta\lambda$ are equivalent with this convention. Under coordinate-adaptive scaling D_t^{-1}, however, putting $\lambda\theta$ inside the adaptive gradient gives

$$\theta_{t+1} = \theta_t - \eta D_t^{-1}(g_t + \lambda\theta_t), \tag{33}$$

so the effective shrinkage depends on the coordinate-wise preconditioner. This is no longer uniform multiplicative weight decay. AdamW decouples shrinkage from the adaptive gradient step, schematically as

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \tag{34}$$

Loshchilov and Hutter [17] made this distinction explicit and evaluated its generalization effects. The mathematics is more important than the name: AdamW corrects a *regularization implementation issue*, not a convergence guarantee for all non-convex problems.

Learning-rate schedules also matter in optimizer comparisons. Warm restarts and cyclical schedules change the sequence of step magnitudes [28], [29]; initialization and normalization change the geometry presented to the optimizer [37], [38]. A comparison that gives one optimizer extensive tuning but not another may be testing a training recipe rather than the update rule itself.

I. Adafactor: Factoring a Large Second-Moment State

An $m \times n$ weight matrix has mn parameters. Adam stores mn first-moment and mn second-moment values, excluding the parameters and gradients themselves. Adafactor [31] reduces the dominant adaptive-state cost by storing row and column statistics of the second moment instead of a dense $m\times n$ array. If V is a nonnegative second-moment matrix, a representative factored approximation based on its row and column sums is

$$\widetilde{V}_{ij} = \frac{r_i c_j}{\sum_k r_k}, \qquad r_i = \sum_j V_{ij}, \quad c_j = \sum_i V_{ij}. \tag{35}$$

This is a rank-one reconstruction of the second-moment scale, not a factorization of the weight matrix or Hessian. Storing $r \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$ requires $m + n$ scalars rather than mn. Its fidelity depends on the matrix's second-moment structure, and practical Adafactor includes additional choices such as clipping and relative-step behavior; dropping momentum is part of its lean-memory configuration [31].

Why does this matter? For a very large matrix, the ratio $(m + n)/(mn) = 1/m + 1/n$ can be small. But the benefit varies with tensor shape: vectors cannot realize the same two-sided factorization, and different implementations keep different auxiliary states. "Sublinear memory" describes the second-moment storage for large matrix dimensions, not zero additional memory overall.

J. Lion: Sign-Based Momentum With One Main State

Lion was discovered through symbolic program search rather than constructed by directly approximating a Hessian [32]. A simplified expression for its momentum-based sign update is

$$c_t = \beta_1 m_{t-1} + (1 - \beta_1)\hat{g}_t, \tag{36}$$

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\,\mathrm{sign}(c_t), \tag{37}$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2)\hat{g}_t, \tag{38}$$

where the two momentum coefficients are used in different parts of the update. Except at zero, each signed coordinate update has magnitude η before weight decay; gradient magnitude influences the sign through the momentum combination rather than acting as a direct step multiplier. The method retains one momentum state rather than Adam's separate m and v arrays. This saving is an accounting identity, while actual runtime and success depend on model size, accelerator efficiency, and hyperparameter choices.

The authors report improvements on particular vision, multimodal, and diffusion-model settings, as well as tasks where gains were small or statistically inconclusive [32]. It would therefore be misleading to say Lion is categorically better than AdamW. One must tune its learning rate and weight decay for the different update magnitude, not copy AdamW settings unchanged.

K. Other Modern First-Order Designs and a Synthesis

Optimizer development has not followed a single monotonic sequence. D-adaptation targets learning-rate selection in specific convex analyses [33]; schedule-free training combines interpolation and averaging to reduce reliance on a stopping-time-dependent schedule [34]; Lookahead separates fast inner updates from a slower averaged state [80]. None of these can be reduced to an assertion that newer methods always converge faster. Moreover, an efficient optimizer is only one part of the training system: throughput depends on batch sizes, communication, precision, and parameter placement [76], [77].

Taken together, first-order methods illustrate an important pattern. Each design stores a different summary of history: SGD stores none beyond the current iterate; momentum stores a vector; Adam stores two vectors;

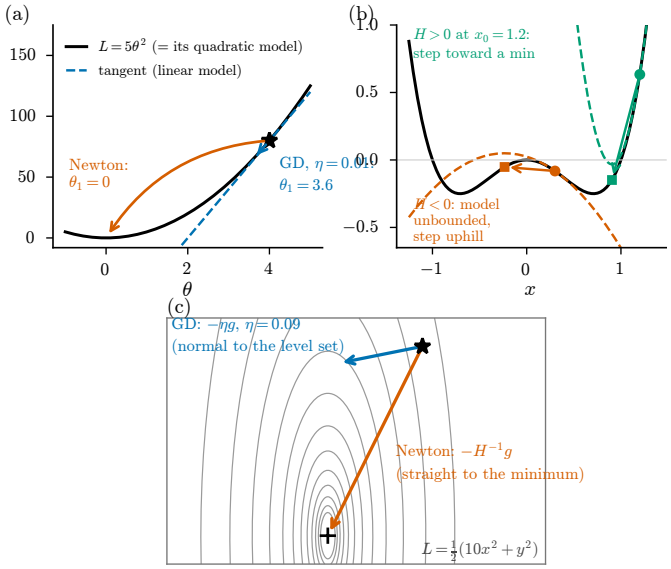


Fig. 6: Newton minimizes a quadratic local model. For an exactly quadratic positive-definite loss, the full step reaches the optimum; for a changing non-convex loss, the quadratic model may be inaccurate or unbounded below.

Adafactor compresses one state for matrices; Lion keeps a momentum vector while using a sign transformation. All use gradients, but their geometry and historical memory differ. Their success on deep learning tasks raises a further question: what additional value can be obtained by considering true curvature or interactions among coordinates? Sections IV and V address that question.

IV. Second-Order Optimization: What Curvature Adds (RQ2)

A. Deriving Newton's Direction From the Quadratic Model

The linear model in Eq. (2) does not predict how the gradient changes after the step. If the Hessian is positive definite in a local region, the second-order Taylor model in Eq. (3) has a unique minimizer. Differentiate the model with respect to Δ:

$$\nabla_\Delta \left(g^\top \Delta + \tfrac{1}{2} \Delta^\top H \Delta \right) = g + H\Delta. \tag{39}$$

Setting this derivative to zero gives the Newton equation

$$H_t \Delta_t = -g_t, \qquad \theta_{t+1} = \theta_t + \Delta_t. \tag{40}$$

For an invertible Hessian, $\Delta_t = -H_t^{-1} g_t$. But numerical algorithms generally solve the linear system instead of constructing the inverse. This distinction is practical: forming an explicit inverse can waste work and magnify numerical error, while matrix factorizations and iterative linear solvers address $H\Delta = -g$ more directly [6], [54].

Worked example. For $L(\theta) = 5\theta^2$, the gradient is $g = 10\theta$ and the Hessian is $H = 10$. Starting from $\theta_0 = 4$, Newton's step is $\Delta = -40/10 = -4$, so $\theta_1 = 0$. GD with $\eta = 0.01$ instead gives $\theta_1 = 4 - 0.01(40) = 3.6$. On this exact quadratic, Newton reaches the minimum in one step because the Taylor model is the loss itself; this special behavior does not extend to general nonlinear losses.

The same holds for a positive-definite quadratic in multiple dimensions,

$$L(\theta) = \tfrac{1}{2} \theta^\top H \theta - b^\top \theta + c, \qquad g = H\theta - b. \tag{41}$$

A full Newton step satisfies $\theta^+ = \theta - H^{-1}(H\theta - b) = H^{-1} b$, the exact quadratic minimizer. In contrast, GD multiplies the error in each Hessian eigenvector direction by $1 - \eta\lambda_j$. Newton corrects directions according to their curvature: for an eigencomponent g_j, the step magnitude is $|g_j|/\lambda_j$ rather than $\eta |g_j|$. This is the essential way second-order optimization differs mathematically from first-order descent.

B. Local Quadratic Convergence and Its Assumptions

The appeal of Newton's method is not only a large step on a quadratic but its local rate near a regular minimizer. Suppose θ^* satisfies $g(\theta^*) = 0$, $H(\theta^*)$ is positive definite, and H is Lipschitz-continuous nearby. For iterates sufficiently close to θ^*, the gradient expansion is

$$g(\theta_t) = H(\theta_t)(\theta_t - \theta^*) + r_t, \qquad \|r_t\| \le C \|\theta_t - \theta^*\|^2. \tag{42}$$

Substituting $\Delta_t = -H(\theta_t)^{-1} g(\theta_t)$ cancels the first-order error term, leaving

$$\|\theta_{t+1} - \theta^*\| \le C' \|\theta_t - \theta^*\|^2. \tag{43}$$

This is *local quadratic convergence*: roughly speaking, once sufficiently close, the number of correct digits can grow rapidly. The estimate depends on nonsingularity, smoothness, and a local neighborhood. It says nothing about a Newton step from a random network initialization, and its per-iteration arithmetic is often much more expensive than GD's [5], [6].

C. Why Undamped Newton Can Fail in Non-Convex Problems

Consider $L(x) = x^4 - x^2$. At $x = 0$, $g(0) = 0$ and $H(0) = -2$. This is a local maximum, not a minimum. More generally, if H has a negative eigenvalue, the quadratic model $g^\top \Delta + \frac{1}{2} \Delta^\top H \Delta$ is unbounded below along its associated direction. Even when H is invertible, the formal Newton direction need not be a descent direction: $g^\top(-H^{-1} g)$ can be positive if H is indefinite. Consequently, basic Newton is not automatically suitable for non-convex neural networks [6], [43].

A simple remedy is damping, replacing H with $H + \lambda I$:

$$(H_t + \lambda_t I)\Delta_t = -g_t. \tag{44}$$

If $H_t + \lambda_t I \succ 0$, the resulting direction is descending whenever $g_t \neq 0$, since $g_t^\top \Delta_t = -g_t^\top (H_t + \lambda_t I)^{-1} g_t < 0$. But choosing λ_t changes the model and may require spectral information or safeguards. A line search can additionally ensure that the actual nonlinear loss decreases, not merely its quadratic approximation. These mechanisms illustrate a central practical fact: curvature information is powerful only when the algorithm controls the accuracy of the curvature model.

D. Trust Regions and Cubic Regularization

Instead of trusting the quadratic model everywhere, a trust-region method constrains its proposed step:

$$\min_{\Delta} \; g_t^\top \Delta + \tfrac{1}{2}\Delta^\top H_t \Delta \tag{45}$$

$$\text{subject to } \|\Delta\| \le r_t. \tag{46}$$

The trust-region radius r_t encodes how far the model is considered reliable. After measuring the ratio of actual loss decrease to predicted model decrease, the method adjusts the radius. This approach accommodates nonconvex curvature more naturally than an unrestricted Newton equation and is developed systematically in classical numerical optimization [6], [54].

Cubic regularization adds a penalty to the second-order model instead:

$$\min_{\Delta} \; g_t^\top \Delta + \tfrac{1}{2}\Delta^\top H_t \Delta + \frac{\rho_t}{3}\|\Delta\|^3. \tag{47}$$

The positive cubic term dominates the indefinite quadratic term for sufficiently large $\|\Delta\|$, so the subproblem is bounded below even when H_t has negative eigenvalues. Nesterov and Polyak established theory for this approach [55], and adaptive cubic regularization developed robust variants [56]. Their rigorous complexity guarantees concern precisely stated smoothness and derivative-access assumptions; full or approximate cubic subproblem solutions may still be expensive for a deep network.

The important conceptual distinction is that trust regions and cubic penalties modify the *model minimization problem*. They are not simply Adam with a different learning rate. They use curvature information and add a mechanism that limits how far the algorithm acts on possibly inaccurate local geometry.

E. BFGS: Learning Curvature From Differences in Gradients

Newton uses H_t directly, requiring Hessian access and a system solve. Quasi-Newton algorithms instead construct an approximation from past changes in parameters and gradients. Define

$$s_t = \theta_{t+1} - \theta_t, \qquad y_t = g_{t+1} - g_t. \tag{48}$$

The mean-value theorem suggests $y_t \approx H s_t$ for small displacements, so s_t, y_t carry information about the Hessian *without explicitly computing it*. If B_t approximates H_t, one seeks a corrected matrix satisfying the secant condition $B_{t+1}s_t = y_t$. Broyden, Fletcher, Goldfarb, and Shanno developed the classical family now abbreviated BFGS [48]–[51].

For $y_t^\top s_t > 0$, one standard BFGS update of the Hessian approximation is

$$B_{t+1} = B_t - \frac{B_t s_t s_t^\top B_t}{s_t^\top B_t s_t} + \frac{y_t y_t^\top}{y_t^\top s_t}. \tag{49}$$

Why this peculiar rank-two correction? Multiplying the right-hand side by s_t cancels the old estimate's prediction $B_t s_t$ and adds the observed change y_t. Thus $B_{t+1}s_t = y_t$, exactly as required. If $B_t \succ 0$ and the curvature condition $y_t^\top s_t > 0$ holds, the update preserves positive definiteness. Then the quasi-Newton step $B_t\Delta_t = -g_t$ is a descent direction. In stochastic training, noisy y_t can violate the secant geometry, motivating damping and regularized stochastic variants.

F. Inverse BFGS and Limited-Memory BFGS

Maintaining B_t is not always convenient; many implementations instead update its inverse approximation $C_t \approx H_t^{-1}$. Let $\rho_t = (y_t^\top s_t)^{-1}$. The inverse-BFGS formula is

$$C_{t+1} = (I - \rho_t s_t y_t^\top) C_t (I - \rho_t y_t s_t^\top) \tag{50}$$

$$+ \rho_t s_t s_t^\top. \tag{51}$$

This version leads to $\Delta_t = -C_t g_t$. Both dense B_t and C_t need $O(d^2)$ memory. When d is the number of neural-network parameters, that is often prohibitive.

Limited-memory BFGS (L-BFGS) avoids forming a dense $d \times d$ matrix. It stores only the latest k pairs (s_i, y_i) and applies an implicit inverse-Hessian approximation through the *two-loop recursion*. The history requires $O(kd)$ numbers rather than $O(d^2)$, and a search direction can be evaluated using vector inner products and scaling. Nocedal's early work and the later Liu–Nocedal analysis established this practical limited-memory design [52], [53]. L-BFGS is thus *curvature-aware through secant observations*, not through explicit second derivatives.

The trade-off deserves precision. A method can save Hessian storage yet still need extra gradient evaluations for line search, additional full-batch consistency to obtain useful curvature pairs, or curvature safeguards. Its favorable behavior on smooth deterministic problems should not be uncritically transferred to noisy minibatch neural-network training. Even the meaning of "one step" differs when one method performs multiple loss evaluations inside that step.

G. Hessian–Vector Products and Hessian-Free Optimization

Dense Hessian storage is not necessary to exploit second derivatives. Given a vector v, a Hessian–vector product is

$$Hv = \frac{d}{d\alpha}\nabla L(\theta + \alpha v)\Big|_{\alpha=0}. \tag{52}$$

Automatic-differentiation techniques can compute this product without materializing H; Pearlmutter provided a classic exact construction [57], and related methods compute curvature-matrix products for neural networks [58]. The cost is generally comparable to a small number of gradient-related passes, rather than $O(d^2)$ storage, although exact implementation cost depends on the framework and model.

A Hessian-free approach seeks an approximate solution to a damped curvature system using an iterative linear solver such as conjugate gradients (CG). Given a positive-definite operator $A \approx H + \lambda I$, CG minimizes the

Algorithm 1 One outer Hessian-free step (schematic)

1: Evaluate gradient g_t at θ_t.
2: Define operator $A_t v = G_t v + \lambda_t v$ through matrix–vector products.
3: Approximately solve $A_t \Delta = -g_t$ using CG.
4: Accept, damp, or rescale Δ using model-quality safeguards.
5: Update $\theta_{t+1} = \theta_t + \Delta$.

quadratic model over successive Krylov subspaces using only products Av. Each inner CG iteration requires such a product, so one outer optimizer step can include many computational passes. Martens applied Hessian-free ideas to deep learning [59] and, with Sutskever, to recurrent networks [60]. The attraction is access to meaningful curvature directions without full matrices; the drawback is variable inner-solve cost and sensitivity to curvature approximations.

Here G_t may be the generalized Gauss–Newton (GGN) matrix instead of the raw Hessian. This matters because the GGN is positive semidefinite for common losses and can produce better-behaved linear systems than the indefinite Hessian; it is not the same matrix in general [58], [65].

H. Hessian, Gauss–Newton, and Fisher: Three Different Matrices

Deep-learning papers sometimes use "second order" loosely. The Hessian $H = \nabla^2 L$ is the second derivative of a chosen scalar objective. For a composite loss $L(\theta) = \ell(f_\theta)$, the chain rule yields

$$H = J^\top H_\ell J + \sum_k \frac{\partial \ell}{\partial f_k} \nabla_\theta^2 f_k. \tag{53}$$

The first term is a generalized Gauss–Newton-type curvature $G = J^\top H_\ell J$, where J is the Jacobian of model outputs with respect to parameters. When $H_\ell \succeq 0$, $G \succeq 0$ even though the full Hessian may be indefinite. Dropping the second term is an approximation, not an identity [65].

For a probabilistic model $p_\theta(y \mid x)$, the Fisher information is

$$F(\theta) = \mathbb{E}_{x,y\sim p_\theta}\left[\nabla_\theta \log p_\theta(y \mid x) \nabla_\theta \log p_\theta(y \mid x)^\top\right]. \tag{54}$$

The distribution in this expectation and the statistical model assumptions matter. Replacing the model-distributed labels with observed data gives an *empirical Fisher* or gradient outer-product estimate; it does not generally equal F, G, or H [63], [64]. In well-specified likelihood problems some identities hold in expectation, but they should never be applied indiscriminately to arbitrary non-convex losses.

I. Natural Gradient: Geometry of Predictions Rather Than Coordinates

Imagine changing parameter coordinates by a smooth, invertible transformation. Ordinary Euclidean GD generally changes its trajectory because Euclidean distance in the new coordinates is different. Natural gradient asks instead for an update that is small in the distributional geometry of the model. Under regularity conditions, the local Kullback–Leibler divergence between nearby predictive distributions satisfies

$$\mathrm{KL}(p_\theta \| p_{\theta+\Delta}) \approx \tfrac{1}{2} \Delta^\top F(\theta) \Delta. \tag{55}$$

Using the Fisher matrix as M in Eq. (12) produces

$$\Delta_{\mathrm{NG}} = -\eta F^{-1} g, \tag{56}$$

where damping or pseudoinversion is needed if F is singular. Amari's natural-gradient framework [61] and later analyses [62], [63] motivate this direction. It is sometimes called second-order because F is matrix-valued and locally encodes distribution curvature, but it is *not simply an inverse Hessian step*. That distinction is the starting point for K-FAC.

J. What the Theory Really Guarantees

On a sufficiently smooth, locally strongly convex problem, full Newton can exhibit local quadratic convergence [6]; BFGS may exhibit local superlinear behavior under appropriate line-search and regularity assumptions [53]. In contrast, the elementary non-convex GD bound in Eq. (18) controls the best gradient norm. Trust-region and cubic-regularization methods admit stronger global complexity results for certain stationarity criteria, but their costs include solving model subproblems [55], [56]. Neither set of results implies that a large neural network is trained faster in wall-clock time or generalizes better. Theoretical guarantees must be read together with derivative-oracle assumptions, local-versus-global scope, and the objective class.

V. Structured Curvature and Modern Matrix-Aware Methods (RQ2)

A. Why Structure Matters in Neural Networks

A layer's weight matrix $W \in \mathbb{R}^{m\times n}$ has mn scalars, but its mathematical role is not an arbitrary list of mn unrelated numbers. Each row corresponds to a set of output interactions and each column to an input feature direction. Flattening W before constructing a diagonal preconditioner loses this two-axis organization. At the other extreme, a dense preconditioner on $\mathrm{vec}(W) \in \mathbb{R}^{mn}$ has $(mn)^2$ elements and becomes rapidly intractable.

Structured methods search for approximations between these extremes. Kronecker products represent certain layer-wise matrices through smaller factors; matrix inverse roots transform row and column directions; and orthogonalization treats a proposed gradient update as a matrix whose singular values can be manipulated. All three exploit tensor structure, but they do *not* estimate the same mathematical object. K-FAC approximates Fisher curvature, Shampoo accumulates gradient second moments in tensor dimensions, and Muon modifies an update's singular spectrum without claiming to estimate either Fisher or Hessian [66], [68], [74].

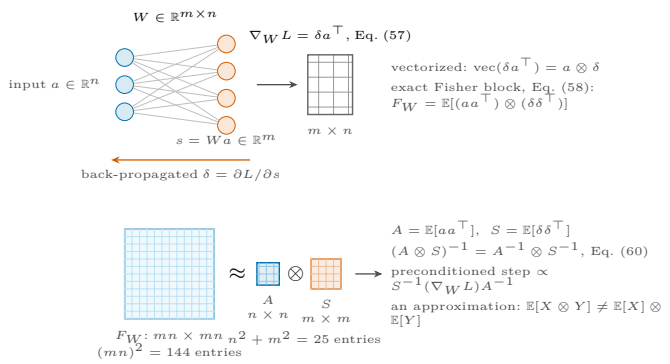


Fig. 7: How K-FAC obtains tractable layer-wise curvature from activation and backpropagated-error statistics. The Kronecker separation is a modeling approximation, not an exact identity for arbitrary data.

B. K-FAC: Approximating Fisher Blocks With Kronecker Products

Consider a fully connected layer $s = Wa$, where $a \in \mathbb{R}^n$ is its input activation and $s \in \mathbb{R}^m$ its preactivation. If $\delta = \partial L/\partial s \in \mathbb{R}^m$, the layer's gradient is

$$\nabla_W L = \delta a^\top. \tag{57}$$

Vectorizing the gradient gives $\mathrm{vec}(\delta a^\top) = a \otimes \delta$ under column-major convention. The corresponding exact Fisher block, schematically, involves

$$F_W = \mathbb{E}[(aa^\top) \otimes (\delta\delta^\top)]. \tag{58}$$

K-FAC makes an approximation that separates this expectation:

$$F_W \approx A \otimes S, \qquad A = \mathbb{E}[aa^\top], \quad S = \mathbb{E}[\delta\delta^\top]. \tag{59}$$

In general, $\mathbb{E}[X \otimes Y] \neq \mathbb{E}[X] \otimes \mathbb{E}[Y]$; the factorization is an approximation, not an algebraic identity. The reward is that for invertible factors,

$$(A \otimes S)^{-1} = A^{-1} \otimes S^{-1}. \tag{60}$$

The dense $mn \times mn$ block has been replaced by separate $n \times n$ and $m \times m$ factors. Their inversion can be amortized and stabilized by damping. On a weight matrix, the corresponding preconditioned gradient has the form $S^{-1}(\nabla_W L)A^{-1}$, up to orientation and damping conventions.

Martens and Grosse [66] introduced K-FAC for efficient natural-gradient-style training, and Grosse and Martens extended the Kronecker-factored approximation to convolutional layers [67]. K-FAC's speed and quality depend on layer structure, the accuracy of the factor approximation, reuse frequency, numerical damping, and communication costs. Its link to statistical information geometry is real, but the approximated Fisher is not the raw parameter Hessian [63], [64].

C. Shampoo: Preconditioning Across Tensor Dimensions

Shampoo approaches matrix structure from a different direction. Instead of approximating a Fisher block, it forms matrix accumulators from gradient outer products. For $G_t \in \mathbb{R}^{m\times n}$, define regularized accumulators

$$L_t = \epsilon I_m + \sum_{s=1}^{t} G_s G_s^\top, \tag{61}$$

$$R_t = \epsilon I_n + \sum_{s=1}^{t} G_s^\top G_s. \tag{62}$$

A representative two-sided matrix Shampoo step uses inverse fourth roots,

$$\Delta W_t = -\eta L_t^{-1/4} G_t R_t^{-1/4}. \tag{63}$$

The matrix powers are spectral functions of positive-definite accumulators: if $L = U\,\mathrm{diag}(\lambda_i)U^\top$, then $L^{-1/4} = U\,\mathrm{diag}(\lambda_i^{-1/4})U^\top$. The two factors jointly rescale the gradient along statistically important row and column directions. Importantly, the exponent in Eq. (63) is for the matrix case; Shampoo's general tensor construction uses related order-dependent exponents and preconditioners [68].

The original Shampoo analysis establishes convergence in a stochastic *convex* setting [68]; that theorem cannot be quoted as a universal non-convex deep-network guarantee. The method avoids a full m^2n^2 block but needs $m^2 + n^2$ factor storage before blocking, and matrix inverse-root computations may be expensive. Distributed and scalable implementations have explored amortizing and parallelizing the matrix operations [69]. The key trade-off is that a mathematically richer direction may need fewer optimizer steps while costing more arithmetic or memory per step.

D. Curvature-Aware Diagonal Alternatives: AdaHessian and Apollo

Full dense curvature is not the only way to exploit second-order information. One class of algorithms estimates diagonal curvature and combines it with an adaptive update. AdaHessian, for example, uses inexpensive stochastic Hessian-diagonal approximations and curvature smoothing to design an adaptive second-order optimizer [70]. Another approach, exemplified by Apollo, learns diagonal quasi-Newton information through parameter and gradient changes rather than constructing the full Hessian [71]. These methods differ both in *what* they estimate (sampled Hessian information versus secant-type updates) and in the guarantees attached to their particular implementations.

A diagonal Hessian approximation reduces memory toward $O(d)$, but it can miss off-diagonal interactions. The rotated quadratic in Section II makes the limitation concrete: even an exact Hessian diagonal need not resolve coupling between rotated principal axes. Conversely, a diagonal method may be preferable when large structured factors create a bottleneck or when noisy estimates of full curvature are unreliable. More elaborate matrices should

be justified by a measurable benefit, not their apparent sophistication.

E. Sophia: Diagonal Hessian Estimation With Clipped Updates

Sophia directly targets efficient curvature-aware language-model pretraining. It maintains momentum estimates and an occasional, lightweight diagonal Hessian estimate, then constructs an elementwise clipped preconditioned update [72]. In simplified notation,

$$\Delta\theta_t = -\eta_t \,\mathrm{clip}\left(\frac{m_t}{\max\{h_t, \epsilon\}}, -\rho, \rho\right), \tag{64}$$

where h_t denotes a stabilized curvature estimate, the maximum and clipping are elementwise, and the full practical algorithm includes particular choices of estimator, update schedule, and weight decay. This schematic emphasizes the two conceptual ingredients: *curvature scaling* and *explicit control of step magnitude*. The denominator is not a squared-gradient moving average as in Adam.

One route to estimating a Hessian diagonal is Hutchinson-style random probing. For a random Rademacher vector $z \in \{-1, 1\}^d$ with independent entries,

$$\hat{h} = z \odot (Hz). \tag{65}$$

Because $\mathbb{E}[z_i z_j] = \mathbf{1}\{i = j\}$, $\mathbb{E}[z_i (Hz)_i] = \sum_j H_{ij}\mathbb{E}[z_i z_j] = H_{ii}$. Thus Eq. (65) is an unbiased estimator of the diagonal of the *Hessian under the stated randomness*, not an unbiased estimate of a positive curvature matrix. Hessian diagonals can be negative in nonconvex regions, so clipping, regularization, and details of the estimator matter. This derivation explains why a method can access some second-order information through Hessian–vector products without storing the Hessian [57], [72].

Sophia's original study reports about a two-fold improvement relative to the Adam baseline in the evaluated GPT pretraining settings spanning 125M–1.5B parameters, counting steps, compute, and wall-clock time to a matched target [72]. That is an *attributed experimental result*, not a guarantee that Sophia halves the cost of every model. Its Hessian-estimation frequency, clipping threshold, precision, and model class all influence the conclusion.

F. SOAP: Adam-Style Adaptation in a Rotated Matrix Basis

SOAP builds on a connection between Shampoo's eigenspaces and familiar adaptive updates. Suppose the left and right Shampoo factors have eigendecompositions $L = U\Lambda U^\top$ and $R = V\Gamma V^\top$. The gradient expressed in their rotated coordinates is

$$\widetilde{G} = U^\top G V. \tag{66}$$

SOAP accumulates Adam-style moment information in this evolving coordinate system, applies an adaptive update in the rotated basis, and transforms it back to the original parameter matrix. This is not merely "Adam plus an SVD"; the update depends on the preconditioner eigenbasis and how it is tracked across steps. The work analyzes a formal connection between particular Shampoo and Adafactor constructions and proposes a more practical rotated-basis adaptive optimizer [73].

In the cited language-model experiments on the particular 360M- and 660M-parameter settings, SOAP's authors report over 40% fewer iterations and over 35% lower wall-clock time compared with AdamW in the large-batch regime [73]. The different percentages are an instructive reminder that faster per-step progress does not translate one-for-one into elapsed-time savings. Matrix decompositions may be updated periodically, and the frequency becomes an optimizer hyperparameter. We cannot generalize these percentages to all architectures, hardware systems, or optimizer implementations.

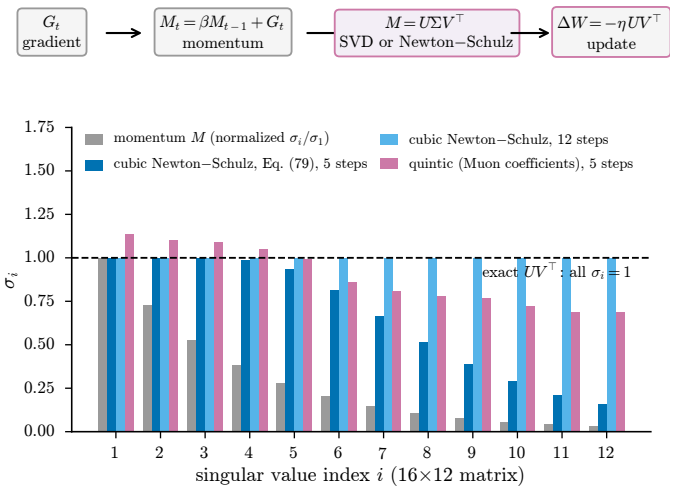


Fig. 8: Matrix geometry in Muon. Orthogonalization changes the singular spectrum of the proposed weight update without estimating the model's Hessian; Newton–Schulz iterations are a numerical device for the matrix operation.

G. Muon: Orthogonalizing a Matrix Update Is Not Newton's Method

Muon is an especially instructive modern case because it uses a matrix-valued transformation but is not a Hessian approximation. The original optimizer applies momentum to hidden-layer weight gradients and then approximately orthogonalizes the resulting update matrix through Newton–Schulz iterations [74]. Let a momentum matrix have singular value decomposition

$$M = U\Sigma V^\top. \tag{67}$$

The semi-orthogonal polar factor is

$$\mathrm{msign}(M) = UV^\top. \tag{68}$$

Where M has nonzero singular values, this transformation replaces their magnitudes by one while retaining left

and right singular directions. Consequently, it treats the matrix's action on directions differently from elementwise normalization or coordinate-wise Adam scaling.

Why might such a direction be useful? If a matrix update Δ is constrained by its *spectral norm*, the linearized loss change is $\langle G, \Delta \rangle_F = \mathrm{tr}(G^\top \Delta)$. Von Neumann's trace inequality yields

$$\min_{\|\Delta\|_2 \le \eta} \langle G, \Delta \rangle_F = -\eta \sum_i \sigma_i(G), \tag{69}$$

attained, for the nonzero singular directions, by $\Delta = -\eta U V^\top$ when $G = U \Sigma V^\top$. In words, the polar factor is steepest descent under a matrix spectral-norm constraint for the *linearized problem*. This is a different geometry from Euclidean steepest descent and a different mechanism from multiplying by H^{-1} [74], [75].

Computing the exact SVD at every update could be expensive. Newton–Schulz iterations approximate matrix orthogonalization using repeated matrix multiplications, commonly with normalization and carefully chosen polynomial coefficients. Their name evokes Newton-like numerical iteration, but *they do not turn Muon into Newton optimization with the neural-network Hessian.* The orthogonalization is applied to the *momentum matrix*, not to the Hessian of the loss. Implementations often pair Muon on suitable hidden weight matrices with AdamW for embeddings, biases, gains, and output heads; not all parameters are eligible for the same matrix operation [74].

H. What Is Being Inverted, Estimated, or Normalized?

It is tempting to put K-FAC, Shampoo, SOAP, Sophia, and Muon in one box labeled *advanced second-order optimization.* That label hides essential differences. K-FAC's matrix derives from Fisher-information blocks; Shampoo's factors derive from accumulated gradient products; SOAP adapts moment estimates in a basis linked to structured factors; Sophia intentionally estimates diagonal Hessian curvature; and Muon approximately orthogonalizes a momentum matrix [66], [68], [72]–[74]. Some methods use an $O(d)$ diagonal, some $O(m^2 + n^2)$ structural factors, and others avoid persistent inverse-root matrices but spend matrix-multiplication work during the update.

There is also a distinction between *normalization* and *preconditioning.* A preconditioner premultiplies a gradient by a matrix or operator intended to rescale directions; a normalization may replace magnitudes with a prescribed norm or sign. Both alter the update, yet they optimize different local models and require different convergence analyses. Modern norm-based interpretations of matrix optimizers make this distinction particularly clear [75].

Figure 9 should therefore compare how each method changes the same synthetic gradient field, not just place their loss curves next to each other.

VI. Critical Comparison: Theory, Geometry, and Cost

A. Comparing the Mathematics Rather Than the Names

The optimizer families can be understood as responses to different bottlenecks. GD has an unbiased full-gradient direction but can move slowly in ill-conditioned regions; SGD saves gradient computation at the cost of noise. Momentum averages directions and may accelerate persistent motion, but it adds a dynamical state and possible overshoot. Adam changes coordinate-wise step scales using past gradients; it can be effective on heterogeneous features, yet its statistics are not curvature estimates and its guarantees depend on assumptions. Newton attempts to solve a local quadratic model using the Hessian; L-BFGS approximates inverse curvature from secant data. K-FAC, Shampoo, and SOAP exploit structure; Sophia uses diagonal Hessian estimates; Muon imposes a matrix-norm geometry [6], [9], [16], [72], [74].

A crucial point is that *all these updates can be written as formulas, but not all formulas explain the same kind of information.* Multiplying g by H^{-1} uses the second derivative of the loss. Multiplying by a diagonal matrix built from squared gradients uses historical stochastic statistics. Replacing singular values by one optimizes a linear model under a spectral-norm budget. These can sometimes produce superficially similar trajectories in a toy problem, yet differ in invariance, noise response, and computational expense.

B. Optimizer-State Memory: Count the Actual Tensors

Table II summarizes representative *persistent* states. The counts deliberately exclude model weights, gradients, activation storage, master weights, and framework bookkeeping. They also exclude transient eigendecomposition buffers and CG workspaces, which matter for peak memory. Numbers are scalar counts, not bytes; multiplying by bytes per stored element requires knowing the implementation's precision. A common half-precision-parameter training setup may still maintain some optimizer states in full precision [76], [77].

For example, an $m \times n$ Adam matrix needs $2mn$ moment scalars; a lean factored Adafactor second moment needs $m+n$; and a two-sided structured factorization uses about m^2+n^2 scalars. It would be *incorrect* to assume that m^2+n^2 is always smaller than mn: for square $m = n$, it is $2n^2$ versus n^2. Structured methods are economical relative to a full $(mn)^2$ layer curvature matrix, not necessarily relative to Adam's diagonal states. This arithmetic distinction is important when claiming that a method "scales" [31], [66], [68].

C. Arithmetic Cost, Communication, and Wall-Clock Time

Per-step cost is not captured by optimizer-state memory. SGD and Adam primarily use elementwise operations after a gradient is available. Newton may require a matrix solve, quasi-Newton methods require vector history

TABLE II: Representative optimizer auxiliary-state complexity. Here d is the total number of parameters, $W \in \mathbb{R}^{m\times n}$ is one matrix, and k is limited-memory history size. Counts exclude parameters, gradients, activations, sharding, master weights, and temporary buffers; they are not a universal peak-memory ranking.

Method	Typical persistent auxiliary state	Order	Main per-step overhead	Important qualification
GD / minibatch SGD	None (apart from iteration control)	$O(d^0)$	One gradient evaluation	Full and minibatch gradients have different data-access costs.
Heavy-ball momentum	Velocity vector	$O(d)$	Vector arithmetic	A momentum buffer can be stored at different precision.
Adam / AdamW	First- and second-moment arrays	$2d$	Elementwise operations	Optional master weights, sharding and implementation buffers are extra.
Lion	One momentum array	d	Elementwise sign and moments	Sign updates change tuning and effective step magnitude.
Adafactor (matrix, lean version)	Row and column second-moment sums	$m+n$	Factored reconstruction	Full configuration may maintain additional states.
Dense Newton	Full Hessian or factorization	$O(d^2)$	Second derivatives / linear solve	Hessian-free variants avoid dense storage but add inner products.
L-BFGS	k recent (s_i, y_i) pairs	$O(kd)$	Two-loop recurrence, possibly line search	Stochastic gradient noise can spoil curvature pairs.
K-FAC (one layer)	Two Fisher factors	$O(m^2+n^2)$	Factor updates, solves	Blocking, damping, inverse refresh, and communication vary.
Shampoo (one matrix)	Row/column accumulators	$O(m^2+n^2)$	Matrix inverse roots and multiplies	Practical implementations amortize and block factor operations.
Sophia	Momentum and diagonal-curvature estimates	$O(d)$	Occasional curvature estimation	Precise number of arrays depends on estimator and implementation.
Muon (eligible matrices)	Momentum, plus temporary workspaces	$O(d)$	Newton–Schulz matrix products	Auxiliary AdamW state for ineligible parameters may remain.

operations, K-FAC and Shampoo maintain and factor matrices, and Muon adds repeated matrix multiplications for update orthogonalization. Conversely, matrix operations can map efficiently to GPU accelerators, while seemingly cheap scalar operations can be memory-bandwidth-limited. Their relative performance depends on shapes, precision, implementation, and hardware [69], [76].

Distributed training adds communication. A curvature factor computed from local minibatches may need averaging or synchronization across workers; parameters and gradient statistics may be sharded; a preconditioner can be updated less frequently to reduce work. Techniques such as ZeRO alter where optimizer states live, so an optimizer's local memory count is not the entire distributed memory budget [77]. Large-model scaling studies make clear why an apparently modest per-step overhead can affect a substantial end-to-end training bill [78], [79].

The fair performance question is therefore not simply *how many iterations reach a target loss?* It is also *how many gradient evaluations, matrix products, accelerator seconds, and memory bytes does that target require?* A curvature-aware method may obtain more decrease per outer step but perform multiple inner CG iterations. A matrix-aware method may need fewer steps but extra matrix root computations. A comparison that reports only the step count can hide these costs. The SOAP authors' distinct iteration and wall-clock improvements are one empirical illustration of this general point [73].

D. Convergence Guarantees Must Match the Problem

Consider four statements that are often juxtaposed: $O(1/T)$ objective error for GD on smooth convex losses; $O(1/T^2)$ objective error for accelerated GD on smooth convex losses; $O(1/T)$ upper bounds on the *best squared gradient norm* in smooth deterministic non-convex GD; and $O(1/\sqrt{T})$ behavior for some stochastic non-convex gradient-norm criteria. They involve different performance measures. A method's non-convex rate may also hide dependence on the gradient noise, smoothness constant, initial optimality gap, or dimension [4], [5], [11].

For instance, the stationarity bound in Eq. (18) implies $T = O(\epsilon^{-2})$ to reach $\|g\| \le \epsilon$ for deterministic GD under its assumptions. Using a stochastic gradient with constant variance and a tuned decreasing step, the bound derived from Eq. (11) often yields an $O(T^{-1/2})$ rate for *expected squared gradient norm*, with constants depending on minibatch size and variance. This suggests $T = O(\epsilon^{-4})$ for achieving an ϵ *gradient norm* by that analysis. These are worst-case statements about different oracle models, not evidence that practical SGD always needs quadratically more time than GD. Adaptive and variance-reduction analyses use their own assumptions [10], [11], [19], [24].

Similarly, local quadratic convergence of Newton concerns an iterate sufficiently close to a nonsingular local minimizer and assumes an appropriate Hessian. It is not a blanket claim that Newton has a globally superior rate on arbitrary non-convex problems. The original Shampoo convex regret guarantee [68], Adam-type non-convex results under extra conditions [19], and local quasi-Newton results [6] cannot be ranked using only the exponent of T. Comparing theorem statements without checking their hypotheses is a category error.

E. Stability, Noise, and Generalization Are Different Questions

The loss surface is dynamic in the practical sense that the Hessian changes with θ. As Eq. (9) shows, even ordinary GD may oscillate when the local curvature exceeds the step-size stability threshold. Recent observations of edge-of-stability behavior emphasize that the global nonlinear training path can continue despite local quadratic instability [42]. It would be wrong to treat the classical $2/\lambda_{\max}$ threshold as a universal prohibition on larger neural-network learning rates. At the same time, large learning rates can cause genuine divergence in other settings. The appropriate statement depends on the local model and training regime.

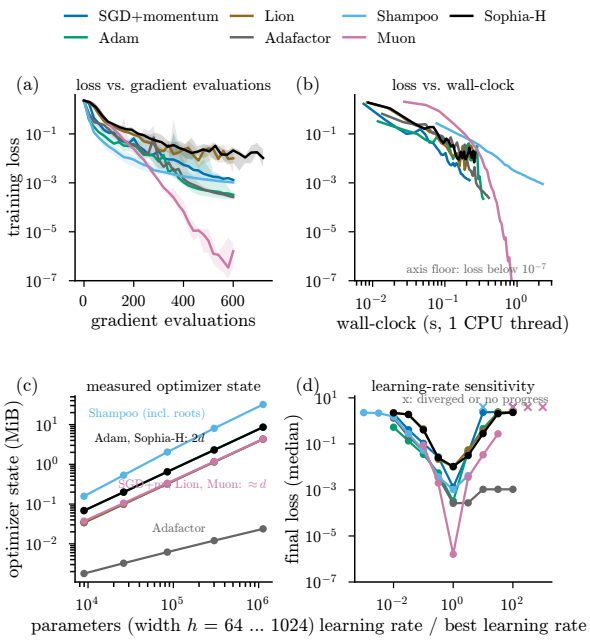
Gradient noise may help exploration around strict saddles, but it can also interfere with precise curvature estimates. Under suitable assumptions, random perturbations enable efficient saddle escape [45], [46]; however, this does not imply all stochastic optimizers avoid all degenerate saddles. Matrix-aware updates and approximate curvature factors introduce additional sensitivity to finite-batch estimates, regularization, and update frequency. Their robustness needs experimental evaluation across batches and initializations rather than extrapolation from a single training run.

Generalization is another separate axis. The relation between sharpness and test performance is complicated by parameterization: rescaling certain network parameters can change Hessian sharpness without changing the represented function [40]. Large-batch/generalization studies [39], comparisons of adaptive and nonadaptive methods [26], [27], and sharpness-aware methods [41] raise interesting issues, but a lower training loss or stronger optimization theorem alone does not establish better test accuracy. We therefore place generalization in the evaluation protocol rather than treating it as a guaranteed output of any optimizer family.

F. A Reproducible Comparison Protocol

For a controlled study, begin with one differentiable toy loss and one small neural-network training task. The toy loss should include both an axis-aligned and a rotated ill-conditioned quadratic; this isolates the effect of diagonal versus full-matrix geometry. The neural-network task should fix the architecture, data split, random initializations, loss, batch size, and training budget. Learning rates, momentum coefficients, damping, preconditioner update frequencies, and weight decay must be tuned with comparable effort. Report at least several random seeds and make the optimizer implementation explicit, including parameter groups and precision.

We recommend four complementary plots, all marked as *proposed experiments* until actually run: (i) loss versus gradient evaluations; (ii) loss versus wall-clock time; (iii) peak accelerator memory versus model size; and (iv) performance versus learning-rate setting. Additionally, show a 2D contour trajectory for GD, momentum, Adam-style diagonal scaling, and exact Newton on the same analytic objective. For Shampoo and Muon, compare matrix preconditioning and orthogonalization on a controlled matrix gradient, without suggesting that a single toy example proves superiority in deep learning.



Measured runs: MLP 64-128-128-10 (ReLU), sklearn digits (1,437 train), batch 64, 600 updates, 5 seeds, fp32, CPU (Intel Core Ultra 9 285H), PyTorch 2.11; 9-point LR grid per optimizer; (a), (b) at each optimizer's best LR.

Fig. 9: Recommended evaluation dashboard. Matching compute, data, initialization, and tuning effort is necessary before interpreting empirical optimizer comparisons.

TABLE III: What each family answers and what it does *not* establish by itself.

Family	Main strength	Not automatically implied
SGD/momentum	Simple, low auxiliary overhead	Fastest wall-clock convergence
Adam/AdamW	Coordinate adaptation	Hessian approximation or global optimum
Adafactor/Lion	Lower auxiliary state in selected regimes	Better loss on every task
Newton/L-BFGS	Local or secant curvature	Stable global non-convex progress
K-FAC	Structured Fisher approximation	Exact Hessian or cheap communication
Shampoo/SOAP	Tensor geometry and rotated adaptation	Lower total memory than all diagonal methods
Sophia	Estimated diagonal Hessian and clipping	Universal reported speedup
Muon	Orthogonalized hidden-weight updates	A second-order Hessian guarantee

G. A Compact Cross-Family Summary

The summary is deliberately qualitative. A more useful conclusion than "Algorithm X wins" is that a method

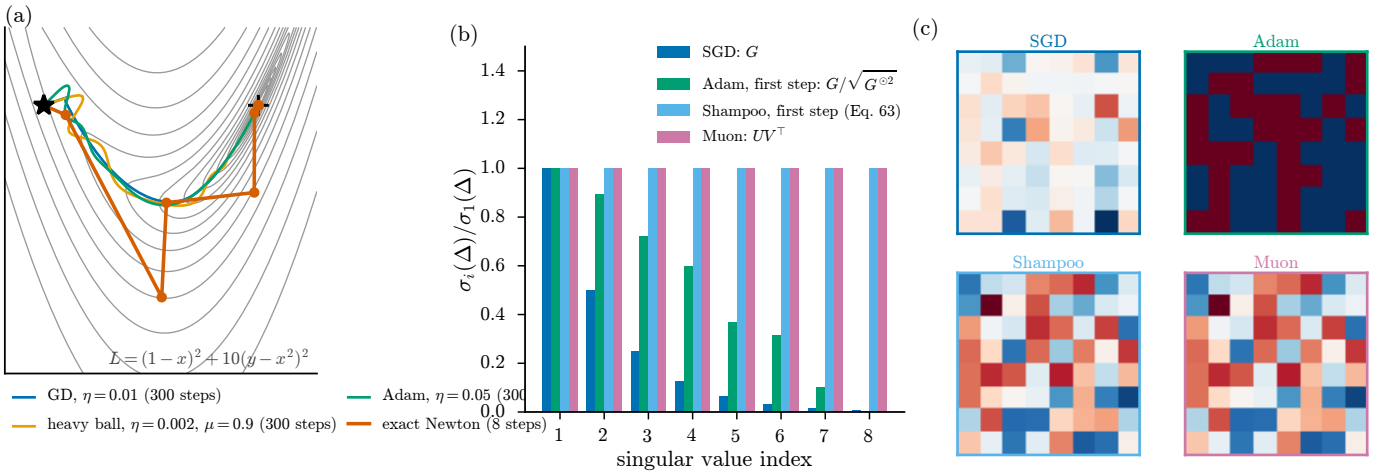


Fig. 10: The analytic comparisons recommended in Section VI-F. (a) GD, heavy-ball momentum, Adam, and exact Newton on one analytic objective, $L = (1-x)^2 + 10(y-x^2)^2$, from the same start $(-1.2, 1)$ (star) to the minimizer $(1, 1)$ (cross); step counts in the legend. (b) Singular values of the update that each method builds from one controlled 8×8 gradient G with singular values $4, 2, 1, \ldots, 0.03$. SGD keeps the spectrum of G; Adam's first step $G/\sqrt{G^{\odot 2}}$ is elementwise and does not flatten it; one Shampoo step from Eq. (63) with no accumulated history equals $UV^\top$, the Muon direction of Eq. (68). (c) The four update matrices (colour scale normalized per matrix). With statistics accumulated over many steps, Shampoo and Muon differ. A toy example; it does not establish superiority in deep learning.

should be chosen by matching its representation of geometry to the scale, parameter structure, data noise, and available compute of the task. A full-Hessian method is theoretically elegant but usually unsuitable for billions of parameters; a diagonal method is economical but can miss rotations; a matrix-aware method can use structural information but may have substantial fixed overhead. Recent results on adaptive and matrix-aware methods are best interpreted as progress in this design space rather than a final ranking [9], [72]–[74].

VII. Open Problems and Future Directions

A. When Does Better Geometry Repay Its Cost?

The main practical question is whether expensive geometric information reduces the number of expensive training operations enough to justify itself. For K-FAC and Shampoo, factor quality, refresh frequency, matrix roots, and distributed synchronization interact. For Hessian-free optimization, the inner CG solve may dominate. For Muon, matrix multiplications may be efficient on suitable accelerators, but eligible matrices and auxiliary parameter groups complicate memory comparisons. Research should therefore report both idealized optimizer arithmetic and complete training-system costs [66], [69], [74], [77].

B. Curvature Under Noise and Non-Convexity

A stochastic Hessian estimate can be indefinite or noisy; an empirical Fisher can differ materially from the true Fisher; a structured gradient covariance may only indirectly reflect loss curvature [64]. It remains important to establish which curvature approximations are sufficiently accurate along real training trajectories, how much damping or clipping is needed, and whether apparent acceleration survives when gradients and curvature factors are estimated from differently sized minibatches. The right benchmark includes noisy, poorly conditioned, and rotated problems, rather than only nearly quadratic convex examples.

C. Matrix-Aware Methods Need Their Own Theory

Muon demonstrates that useful optimizer geometry need not be an estimated inverse Hessian. The spectral-norm interpretation explains one mathematical aspect of its update, but leaves open the effects of momentum, nonlinearity, approximate orthogonalization, stochastic gradients, and changing layer shapes. Theoretical comparison with Shampoo and SOAP should separate their actual operators and not relabel all matrix transforms as second order. Another challenge is understanding which parameter blocks benefit from matrix-aware updates, as empirical implementations frequently retain AdamW for embeddings and nonmatrix parameters [73]–[75].

D. Invariance and Parameterization

An optimizer's trajectory may change when the same predictive model is expressed in different coordinates. Euclidean and diagonal methods use parameter coordinates directly; natural gradient attempts a geometrically meaningful transformation of predictive distributions; structured and matrix-aware methods introduce different invariances or biases. These properties matter for networks with normalization, scale symmetries, and large variation in layer width [38], [40], [61], [63]. A promising direction is

TABLE IV: Update rules of the surveyed optimizers in one notation. Equation numbers refer to this survey; "state" lists what persists between steps (compare Table II). The last column names what the state represents, following Sections III–V.

Method	Update (one step)	Persistent state	Main hyperparameters	What the state represents
GD / SGD	$\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)$ or $\theta_t - \eta_t \hat{g}_t$ (14), (15)	none	η_t, batch size	—
Heavy ball	$v_t = \mu v_{t-1} + g_t,\ \theta_{t+1} = \theta_t - \eta v_t$ (19), (20)	v	η, μ	discounted sum of past gradients
Nesterov	$y_t = \theta_t + \mu(\theta_t - \theta_{t-1}),\ \theta_{t+1} = y_t - \eta \nabla L(y_t)$ (23), (24)	previous iterate	η, μ	look-ahead point (first-order)
AdaGrad	$s_{t,j} = s_{t-1,j} + \hat{g}_{t,j}^2$, $\theta_{t+1,j} = \theta_{t,j} - \eta \hat{g}_{t,j}/(\sqrt{s_{t,j}} + \varepsilon)$ (25), (26)	s	η, ε	cumulative squared gradients
RMSProp	$s_t = \rho s_{t-1} + (1-\rho)\hat{g}_t^{\odot 2}$, $\theta_{t+1} = \theta_t - \eta \hat{g}_t/(\sqrt{s_t} + \varepsilon)$ (27)	s	η, ρ, ε	exponential average of squared gradients
Adam	m_t, v_t by (28), (29); $\theta_{t+1} = \theta_t - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$ (30)	m, v	$\eta, \beta_1, \beta_2, \varepsilon$	first moment (direction) and second raw moment (scale), not diag H
AMSGrad	$\bar{v}_t = \max(\bar{v}_{t-1}, v_t),\ \Delta_t \propto -m_t/(\sqrt{\bar{v}_t} + \varepsilon)$ (31)	$m, v, \bar{v}$	as Adam	running maximum of the second moment
AdamW	$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$ (34)	m, v	as Adam, $+\lambda$	weight decay decoupled from the adaptive scale
Adafactor	second moment replaced by $\widetilde{V}_{ij} = r_i c_j / \sum_k r_k$ (35)	$r \in \mathbb{R}^m$, $c \in \mathbb{R}^n$	η, decay, clipping	rank-one second-moment scale
Lion	$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\, \mathrm{sign}(\beta_1 m_{t-1} + (1-\beta_1)\hat{g}_t)$ (36)–(38)	m	$\eta, \beta_1, \beta_2, \lambda$	momentum; only its sign reaches the update
Newton	$H_t \Delta_t = -g_t,\ \theta_{t+1} = \theta_t + \Delta_t$ (40)	(H or factorization)	damping, line search	exact local curvature
Damped / trust region / cubic	$(H_t + \lambda_t I)\Delta_t = -g_t$ (44); model (45)–(47) with $\|\Delta\| \le r_t$ or $+\frac{\rho_t}{3}\|\Delta\|^3$	—	λ_t, r_t, ρ_t	curvature plus a safeguard on the step
BFGS / L-BFGS	B_{t+1} by (49) or C_{t+1} by (50), (51); L-BFGS: two-loop recursion	B or C; k pairs (s_i, y_i)	k, line search	curvature from secant pairs $y_t \approx H s_t$
Hessian-free	approximately solve $(G_t + \lambda_t I)\Delta = -g_t$ by CG (Alg. 1)	—	CG iterations, λ_t	products Hv (52) or Gv, no stored matrix
Natural gradient	$\Delta_{\mathrm{NG}} = -\eta F^{-1} g$ (56)	(F)	η, damping	Fisher metric on predictive distributions
K-FAC	$F_W \approx A \otimes S$; step $\propto S^{-1}(\nabla_W L)A^{-1}$ (59), (60)	A, S per layer	η, damping, refresh	Kronecker-factored Fisher block
Shampoo	$\Delta W_t = -\eta L_t^{-1/4} G_t R_t^{-1/4}$ (61)–(63)	L, R per matrix	η, ϵ, refresh	accumulated $GG^\top$, $G^\top G$ (gradient statistics)
SOAP	Adam-style moments of $\widetilde{G} = U^\top G V$, update rotated back (66)	moments, U, V	as Adam, basis refresh	moments in Shampoo's eigenbasis
Sophia	$\Delta\theta_t = -\eta_t\, \mathrm{clip}(m_t/\max\{h_t, \epsilon\}, -\rho, \rho)$ (64)	m, h	η, ρ, estimate interval	diagonal Hessian via $z \odot (Hz)$ (65)
Muon	$M = U\Sigma V^\top,\ \Delta W = -\eta\, \mathrm{msign}(M) = -\eta U V^\top$ (67)–(69)	M (momentum)	η, β, Newton–Schulz steps	no curvature: singular values set to one

to measure such invariances empirically without assuming that more invariance must always improve generalization.

E. Toward Fair, System-Aware Optimizer Benchmarks

Optimizer studies should state exactly what is counted. Number of updates, examples processed, tokens processed, gradient evaluations, GPU-hours, peak memory, communication volume, and final held-out performance are related but not interchangeable. Training-scale studies [78], [79] make these trade-offs central. Public comparisons will be more informative if optimizer implementations, hyperparameter searches, mixed precision, parameter group exclusions, and hardware assumptions are released alongside results.

VIII. Conclusion

Optimization methods for deep learning can be understood as different ways of interpreting one object: the gradient of training loss. GD follows it in Euclidean geometry; SGD estimates it from a minibatch; momentum accumulates its history; Adam and Adafactor use coordinate-dependent gradient statistics; Newton and quasi-Newton methods incorporate loss curvature; natural gradient and K-FAC use Fisher-information geometry; Shampoo and SOAP use structured matrix statistics; Sophia estimates diagonal Hessian curvature; and Muon reshapes a momentum update according to matrix singular directions. Those mechanisms are related through local optimization, yet they are not interchangeable.

The answer to **RQ1** is that first-order methods progressively trade simple, memory-light updates for historical and coordinate-wise information, with improvements that depend on tuning, stochasticity, and model class. The answer to **RQ2** is that second-order, curvature-aware, and matrix-aware methods can better reflect anisotropy or structure, but they pay for matrix information, approximation errors, additional hyperparameters, or restricted parameter types. The central lesson is methodological: distinguish Hessians from gradient moments, theorems from observed speedups, and update counts from full training costs. These distinctions provide a foundation for understanding current optimizer research without treating the newest algorithm as automatically the best.

TABLE V: Convergence and stability statements quoted in this survey, with the setting each one requires. Statements in different rows measure different quantities and cannot be ranked by their exponent of T alone (Section VI-D).

Statement	Objective class and assumptions	Success criterion	Oracle per step	Where
GD: $O(1/T)$	smooth convex	function error	full gradient	Sec. VI-D [5]
Accelerated GD: $O(1/T^2)$	smooth convex, appropriate coefficient schedule	function error	full gradient	Sec. III-C [4], [5]
Fixed-step GD rate governed by κ	strongly convex quadratic	error per step	full gradient	Sec. II-D [5], [6]
GD: $\min_t \|g_t\|^2 \le 2[L(\theta_0) - L_*]/(\eta T)$, i.e. $O(\epsilon^{-2})$	β-smooth, bounded below, $\eta \le 1/\beta$	best squared gradient norm	full gradient	Eq. (18)
SGD: $O(T^{-1/2})$, i.e. $O(\epsilon^{-4})$	β-smooth, bounded below, unbiased, variance $\le \sigma^2/B$	average expected squared gradient norm	minibatch gradient	Eqs. (75)–(78) [11]
Heavy ball stable iff $0 < \eta\lambda < 2(1+\mu)$	one-dimensional quadratic, $\lambda > 0$, $0 \le \mu < 1$	local contraction	full gradient	Eq. (22)
Preconditioned GD stable iff $0 < \eta < 2/\lambda_{\max}(K)$	fixed positive-definite quadratic and fixed positive-definite M	contraction of every eigenmode	gradient and fixed M	Eq. (87)
Newton: local quadratic convergence	near a nonsingular local minimizer, Lipschitz Hessian	distance to θ^*	Hessian and linear solve	Eq. (43) [6]
BFGS: local superlinear behavior	appropriate line search and regularity	distance to θ^*	gradients (secant pairs)	Sec. IV-J [53]
Trust region, cubic regularization: global complexity	precisely stated smoothness and derivative access	certain stationarity criteria	Hessian information and a subproblem solve	Sec. IV-D [55], [56]
AdaGrad	online / stochastic convex	regret	stochastic gradient	Sec. III-D [13]
Adam can fail; AMSGrad fix	a simple online convex setting	convergence	stochastic gradient	Sec. III-G [18]
Adam-type non-convex results	extra conditions: step-size variation, moment schedules, or bounded gradients	stationarity	stochastic gradient	Sec. III-G [19], [20], [25]
Shampoo	stochastic convex	regret	gradient and matrix roots	Sec. V-C [68]
Perturbed gradient methods escape strict saddles	suitable assumptions of the cited analyses	approximate second-order stationarity	gradient and random perturbation	Sec. II-E [45], [46]
Muon's polar factor is spectral-norm steepest descent	linearized loss, budget $\|\Delta\|_2 \le \eta$	one-step linear decrease (not convergence)	gradient matrix	Eq. (69) [74], [75]

TABLE VI: Matrices that the literature calls "curvature," and what each one is (Sections IV-H and V-H). Sizes are for one $m \times n$ weight matrix with mn parameters.

Matrix	Definition	Positive semidefinite?	Relation to the Hessian	Stored size	Used by
Hessian H	$\nabla^2 L$; decomposed in Eq. (53)	no: indefinite near saddles	itself	$(mn)^2$	Newton, trust region, cubic, Hv in Hessian-free
Gauss–Newton G	$J^\top H_\ell J$, first term of Eq. (53)	yes when $H_\ell \succeq 0$	H without the second term of Eq. (53): an approximation	$(mn)^2$	Hessian-free (Gv) [65]
Fisher F	$\mathbb{E}_{p_\theta}[\nabla \log p \nabla \log p^\top]$, Eq. (54)	yes	related to G, H only through identities in well-specified likelihood problems, in expectation	$(mn)^2$	natural gradient, K-FAC
Empirical Fisher	gradient outer products with observed labels	yes	does not generally equal F, G, or H [64]	$(mn)^2$	gradient outer-product estimates
K-FAC factors	$A \otimes S$, $A = \mathbb{E}[aa^\top]$, $S = \mathbb{E}[\delta\delta^\top]$, Eq. (59)	yes	approximates the Fisher block F_W, not H	$n^2 + m^2$	K-FAC
Shampoo factors	L_t, R_t of Eqs. (61), (62)	yes (positive definite)	gradient statistics, not loss curvature	$m^2 + n^2$	Shampoo, SOAP
Adam's v	exponential average of $\hat{g}^{\odot 2}$, Eq. (29)	yes (nonnegative diagonal)	$\mathbb{E}[g_j^2]$-like; not $\partial^2 L/\partial\theta_j^2$	mn	Adam, AdamW
Sophia's h	smoothed $z \odot (Hz)$, Eq. (65)	no: entries can be negative	unbiased estimate of diag H	mn	Sophia (clipped)
(Muon)	no metric stored; $\operatorname{msign}(M) = UV^\top$, Eq. (68)	—	spectral-norm geometry, generally not a quadratic metric	mn (momentum)	Muon

Appendix A
Worked Mathematical Connections

A. Diagonal Scaling and a Rotated Quadratic

For $L(\theta) = \frac{1}{2}\theta^\top H\theta$ and diagonal positive definite D, the preconditioned step $\theta^+ = \theta - \eta D^{-1}H\theta$ corresponds, after the change of variables $z = D^{1/2}\theta$, to $z^+ = (I - \eta D^{-1/2}HD^{-1/2})z$. Thus convergence is governed by the *symmetrically preconditioned* matrix $D^{-1/2}HD^{-1/2}$. If H is diagonal and $D = H$, a single unit step solves the quadratic. If H has off-diagonal couplings, merely choosing $D = \operatorname{diag}(H)$ need not remove them. This formalizes the geometric diagrams in Fig. 3 [6], [13].

For a concrete 45° rotation, let

$$Q = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \qquad H = Q^\top \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix} Q. \tag{70}$$

Direct multiplication gives $H_{11} = H_{22} = 50.5$ and off-

diagonal entries with absolute value 49.5. The diagonal contains no information about the large coupling. This example shows why full or structured transformations can offer information unavailable to purely diagonal methods, even though they need more computation.

B. Bias Correction as a Finite Geometric Series

With constant gradient g and $m_0 = 0$, the first-moment recursion $m_t = \beta m_{t-1} + (1-\beta)g$ gives

$$m_t = (1-\beta)\sum_{k=0}^{t-1}\beta^k g = (1-\beta^t)g. \tag{71}$$

The correction $\hat{m}_t = m_t/(1-\beta^t)$ therefore returns g in this idealized stationary case. For changing stochastic gradients, the interpretation is more subtle, but the geometric-series calculation explains why Adam needs separate corrections for its two exponential averages [16].

C. The Polar Factor and the Nuclear Norm

Write $G = U\Sigma V^\top$. For any update Δ with $\|\Delta\|_2 \le \eta$, trace duality gives $|\langle G, \Delta\rangle_F| \le \|G\|_*\|\Delta\|_2$, where $\|G\|_* = \sum_i \sigma_i(G)$ is the nuclear norm. Choosing $\Delta = -\eta UV^\top$ attains the lower bound along the singular directions, showing why the polar factor is the steepest *linearized* direction for a spectral-norm budget. This does not establish a global neural-network convergence theorem for approximate Newton–Schulz-orthogonalized momentum [74], [75].

Appendix B
Proposed Figure and Reproducibility Checklist

All boxed figure slots in this document are intentional blanks. Figures 1–9 specify the concepts to illustrate; they contain no claimed experimental output. Before submission with finalized figures, identify whether each panel is (i) an analytic mathematical plot, (ii) a replication of a published figure with proper attribution, or (iii) a new experiment. Analytic bowls, rotated quadratics, momentum trajectories, SVD singular-value plots, and theoretical memory curves can be generated from their explicitly stated equations. Empirical neural-network training curves must come from actual code runs and should report architecture, dataset, seeds, learning rates, precision, hardware, stopping criteria, and optimizer implementation.

For plots of optimization loss, also specify whether the horizontal axis represents parameter updates, complete data passes, examples/tokens processed, gradient evaluations, or wall-clock time. For memory figures, indicate whether the values include weights, gradients, master weights, optimizer state, factor buffers, activation checkpoints, and distributed sharding. Without these labels, a visual comparison can be technically correct in isolation but misleading to the reader.

Appendix C
Additional Worked Derivations and Practical Diagnostics

A. Step-by-Step Comparison on a Rotated Valley

Consider again the rotated quadratic $L(\theta) = \theta^\top H\theta/2$ with

$$H = \begin{bmatrix} 50.5 & -49.5 \\ -49.5 & 50.5 \end{bmatrix}, \qquad \theta_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \tag{72}$$

Its gradient at the starting point is $g_0 = H\theta_0 = (50.5, -49.5)^\top$ and the loss is $L(\theta_0) = 25.25$. All methods see the *same gradient*. Their different steps reveal what is added by the optimizer, not what is added by the objective.

First, fixed-step GD with $\eta = 0.01$ produces

$$\theta_1^{\mathrm{GD}} = \theta_0 - 0.01 g_0 = \begin{bmatrix} 0.495 \\ 0.495 \end{bmatrix}. \tag{73}$$

Direct substitution gives $L(\theta_1^{\mathrm{GD}}) = 0.5(0.495, 0.495)H(0.495, 0.495)^\top = 0.245025$. The GD step happens to move close to the low-curvature eigenvector because the chosen learning rate cancels the large-curvature eigencomponent. This is a specific, calculated example, not a claim that GD always makes such progress in one step.

Next, use the *exact diagonal of the Hessian* as a preconditioner: $D = \mathrm{diag}(H) = 50.5I$. At learning rate $\eta = 1$, the update becomes

$$\theta_1^{\mathrm{diag}} = \theta_0 - D^{-1}g_0 = \begin{bmatrix} 0 \\ 49.5/50.5 \end{bmatrix} \approx \begin{bmatrix} 0 \\ 0.9802 \end{bmatrix}. \tag{74}$$

Even though D contains the *exact* diagonal curvature, it discards the off-diagonal interaction -49.5 and therefore does not solve the problem. The loss is about 24.26, only modestly below the starting 25.25. In contrast, full Newton solves $H\Delta = -g_0$, so $\Delta = -\theta_0$ and $\theta_1^{\mathrm{Newton}} = 0$. This numerical contrast explains precisely why a diagonal Hessian or adaptive coordinate scale is not equivalent to full inverse-Hessian preconditioning. It also shows that no one-step ranking is meaningful without a fair selection of learning rates for each method [6], [13].

The same construction can be used to generate a useful figure: draw the ellipses of L, mark the common start, and show one step under GD, diagonal scaling, and Newton. Adding arrows for their subsequent steps makes the effect of conditioning visible. Every plotted quantity is determined by the explicit 2×2 matrix above and requires no empirical neural-network training.

B. An SGD Bound With an Explicit Step-Size Choice

Let $\Delta_0 = L(\theta_0) - L_*$, and assume L is β-smooth, bounded below, and minibatch gradients are conditionally unbiased with conditional variance at most σ^2/B. For a constant step $\eta \le 1/\beta$, Eq. (11) simplifies to

$$\mathbb{E}_t[L_{t+1}] \le L_t - \frac{\eta}{2}\|g_t\|^2 + \frac{\beta\eta^2\sigma^2}{2B}. \tag{75}$$

TABLE VII: Provenance of every figure in this version, in the terms of Appendix B: (i) analytic mathematical plot, (ii) replication of a published figure, or (iii) new experiment. No figure is a replication. Scripts are in the survey's repository folder `survey-v3/`.

Fig.	Type	Generated from	Script
1	diagram	the organization of Sections III–V; no data	`tex/fig1_map.tex`
2	(i) analytic	quadratics $\frac{1}{2}\theta^\top H\theta$ with $H = 2I$, diag$(16, 1)$, its 45° rotation, diag$(1, -1)$; exact gradients and eigenvectors	`figs/make_figs.py`
3	(i) analytic	Eqs. (8), (70); GD, diagonal, and Newton steps of Eq. (85)	`figs/make_figs.py`
4	(i) analytic	Eqs. (19)–(22); panel (a) adds seeded Gaussian gradient noise, stated in the panel	`figs/make_figs.py`
5	diagram	Eqs. (28)–(30); state counts of Table II	`tex/fig5_adam.tex`
6	(i) analytic	worked examples of Sections IV-A and IV-C; Eq. (40)	`figs/make_figs.py`
7	diagram	Eqs. (57)–(60), blocks drawn to scale for $m = 4$, $n = 3$	`tex/fig7_kfac.tex`
8	(i) analytic	SVD of a seeded 16×12 matrix; Eq. (79); quintic coefficients of the Muon implementation [74]	`figs/make_figs.py`
9	(iii) new experiment	measured training runs; full record in Table VIII	`experiments/run_dashboard.py`, `plot_dashboard.py`
10	(i) analytic	Rosenbrock-type objective; one controlled matrix gradient; Eqs. (30), (63), (68)	`figs/make_figs.py`
11–14	(i) analytic	Appendices C-A, C-B (illustrative constants), C-C, D-A	`figs/make_figs.py`

TABLE VIII: Run record for the experiment in Fig. 9, item by item as Appendix B requires.

Item	Setting
Architecture	MLP 64–128–128–10, ReLU, PyTorch default initialization
Dataset	scikit-learn digits (8×8 images, 10 classes); fixed split of 1,437 training and 360 held-out images
Loss, batch, budget	cross-entropy; minibatch 64; 600 updates (about 27 epochs); no schedule, no weight decay
Seeds	5 per setting (initialization and minibatch order)
Learning rates	9-point grid per optimizer, half-decade spacing over four decades; panels (a)–(b) at the grid point with the lowest median final loss
Optimizers	SGD with momentum 0.9; Adam (0.9, 0.999); Lion (0.9, 0.99); Adafactor (PyTorch defaults); Shampoo as in Eq. (63), $\epsilon = 10^{-4}$, exact roots each step, no grafting, diagonal AdaGrad for biases; Muon on the two hidden matrices (Nesterov momentum 0.95, five quintic Newton–Schulz steps) with Adam ($\eta = 2 \cdot 10^{-3}$) for biases and output layer; Sophia-H (Hutchinson estimate every 10 steps, $\gamma = 0.01$, $\rho = 1$)
Precision, hardware	fp32; CPU, Intel Core Ultra 9 285H; PyTorch 2.11
Stopping criterion	fixed budget of 600 updates; runs whose loss became non-finite or exceeded 10^3 were stopped and marked diverged
Horizontal axes	(a) gradient evaluations, a Hessian–vector product counted as two; (b) optimizer wall-clock on one CPU thread, evaluation excluded, median of three repeats
Memory counted	persistent optimizer-state tensors only (fp32 bytes), measured after two updates; weights, gradients, and activations excluded; Shampoo's cached inverse roots included

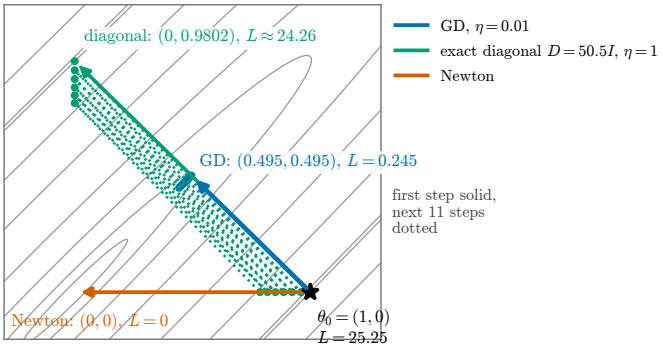


Fig. 11: The construction proposed in Appendix C-A, drawn from the 2×2 matrix of Eq. (72). All three methods start at $\theta_0 = (1, 0)$ with the same gradient $g_0 = (50.5, -49.5)^\top$. Solid arrows: the first steps of Eqs. (73) and (74) and the Newton step; dotted: the next 11 steps. The level sets include the losses quoted in the text, 25.25, 24.26, and 0.245. The exact diagonal preconditioner bounces across the valley because its multipliers are ± 0.98 (Table IX).

Take full expectations, sum for T iterations, use the lower bound L_*, and divide by $\eta T/2$. The result is

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|g_t\|^2 \le \frac{2\Delta_0}{\eta T} + \frac{\beta\eta\sigma^2}{B}. \tag{76}$$

The first term decreases when the step size grows; the second increases. If we temporarily ignore the constraint $\eta \le 1/\beta$, differentiate the right-hand side with respect to η:

$$-\frac{2\Delta_0}{\eta^2 T} + \frac{\beta\sigma^2}{B} = 0. \tag{77}$$

Thus a balanced choice is

$$\eta_{\text{bal}} = \sqrt{\frac{2\Delta_0 B}{\beta\sigma^2 T}}, \qquad \eta = \min\left\{\frac{1}{\beta}, \eta_{\text{bal}}\right\}. \tag{78}$$

Substitution into Eq. (76) gives a bound of order $T^{-1/2}$ in the stochastic regime for the *average expected squared gradient norm*, provided the bounded-variance and independence assumptions are valid. This is why an *ϵ-gradient-norm* objective leads, through this particular worst-case bound, to an $O(\epsilon^{-4})$ iteration dependence. It is not an empirical speed prediction and should not be confused with an $O(1/T)$ convex function-value statement [9], [11].

C. How a Matrix Iteration Can Approximate Orthogonalization

An ideal polar factor for a full-rank matrix is $UV^\top$ when $M = U\Sigma V^\top$. For intuition, consider a normalized

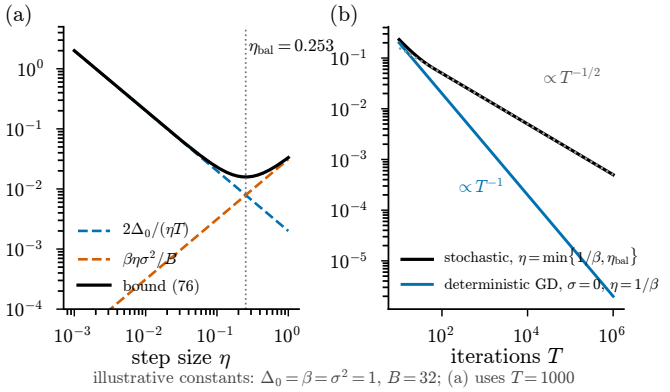


Fig. 12: The bound of Eq. (76) with illustrative constants. (a) Its two terms as functions of η: the balanced choice of Eq. (78) sits where they cross. (b) The resulting worst-case bound versus T: order $T^{-1/2}$ with noise, against the deterministic order T^{-1} of Eq. (18). These are upper bounds, not predicted training curves.

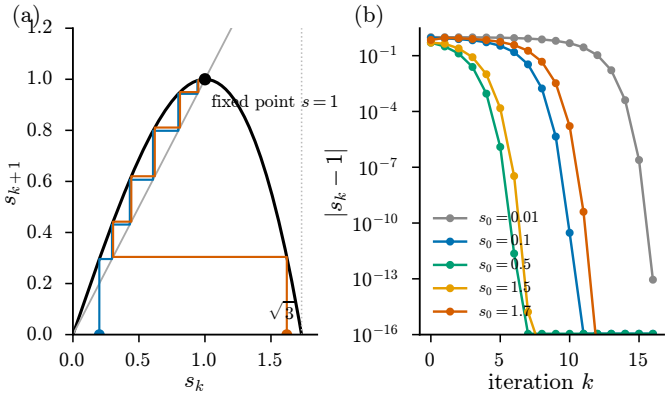


Fig. 13: The scalar Newton–Schulz map of Eq. (80). (a) Cobweb iterations from $s_0 = 0.2$ and $s_0 = 1.62$: both are drawn to the fixed point $s = 1$, where the map has zero slope. (b) Distance to one: slow growth while s_k is small (a factor near 3/2 per step), then rapid convergence once s_k is near one. Starting values must lie below $\sqrt{3}$ for this basic iteration.

matrix X_0 whose nonzero singular values lie in the convergence range of a basic Newton–Schulz polar iteration. One schematic iteration is

$$X_{k+1} = \tfrac{1}{2} X_k (3I - X_k^\top X_k), \tag{79}$$

shown for a compatible matrix shape and mathematical normalization. If $X_k = U\,\mathrm{diag}(s_i)V^\top$ with fixed singular directions, each singular value evolves according to

$$s_{i,k+1} = \tfrac{1}{2} s_{i,k}(3 - s_{i,k}^2). \tag{80}$$

The fixed value $s = 1$ is attractive locally because the derivative of the scalar map is $\frac{3}{2}(1-s^2)$, which vanishes at $s = 1$. Hence errors in singular values can shrink rapidly once they are close to one. This is the basic numerical reason matrix-multiplication iterations can approximate polar factors without computing full singular-value decompositions. Practical Muon implementations use their own polynomial coefficients, scaling, iteration counts, and numerical safeguards [74], [75]; Eq. (79) is a *didactic example*, not a verbatim statement of Muon's production implementation.

An important caveat concerns zero or very small singular values. A polar factor for a rank-deficient matrix is not uniquely determined outside the supported subspace, and numerical iterations require care near zero. Furthermore, approximating the polar factor of a momentum update is an operation on *that update matrix*; it does not compute the Hessian of the neural-network loss. This is why the name "Newton–Schulz" should not cause Muon to be counted as a classical Newton optimizer.

D. Deriving Natural Gradient From a Small-KL Constraint

Ordinary GD asks which Euclidean move has the greatest first-order decrease. Natural gradient instead restricts how much the model distribution may change. Near θ, approximate the KL change by $\frac{1}{2}\Delta^\top F \Delta$. Then solve

$$\min_{\Delta} g^\top \Delta \quad \text{subject to} \quad \tfrac{1}{2}\Delta^\top F \Delta \le r^2/2. \tag{81}$$

When $F \succ 0$ and the constraint is active, the Lagrangian is $\mathcal{L}(\Delta, \lambda) = g^\top \Delta + \frac{\lambda}{2}(\Delta^\top F \Delta - r^2)$. Stationarity yields $g + \lambda F \Delta = 0$, so $\Delta = -\lambda^{-1} F^{-1} g$. The radius determines the multiplier λ, while the *direction* is $-F^{-1}g$. This gives an intuitive derivation of the natural gradient and shows why its goal is distinct from minimizing a Hessian quadratic model [61], [63].

Appendix D
Non-Convex Geometry: When Additional Curvature Can Mislead

A. A Saddle Where Newton Reaches the Wrong Destination

Consider the two-dimensional loss

$$L(x,y) = \frac{1}{2}x^2 - y^2, \qquad \nabla L = \begin{bmatrix} x \\ -2y \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix}. \tag{82}$$

The origin satisfies $\nabla L(0,0) = 0$, but it is a strict saddle because the y direction has negative curvature. Start at $(x_0, y_0) = (1, 0.1)$. A GD step with learning rate η produces

$$(x_1, y_1) = ((1-\eta)x_0, (1+2\eta)y_0). \tag{83}$$

The x component contracts when $0 < \eta < 2$, while the y component expands because $1 + 2\eta > 1$. In this local quadratic example, negative curvature is precisely what pushes GD away from the saddle, provided initialization has a nonzero component in that direction [43], [44].

Now take the full, unrestricted Newton step. Since $H^{-1} = \mathrm{diag}(1, -1/2)$, we obtain

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} - H^{-1}\nabla L(x_0, y_0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \tag{84}$$

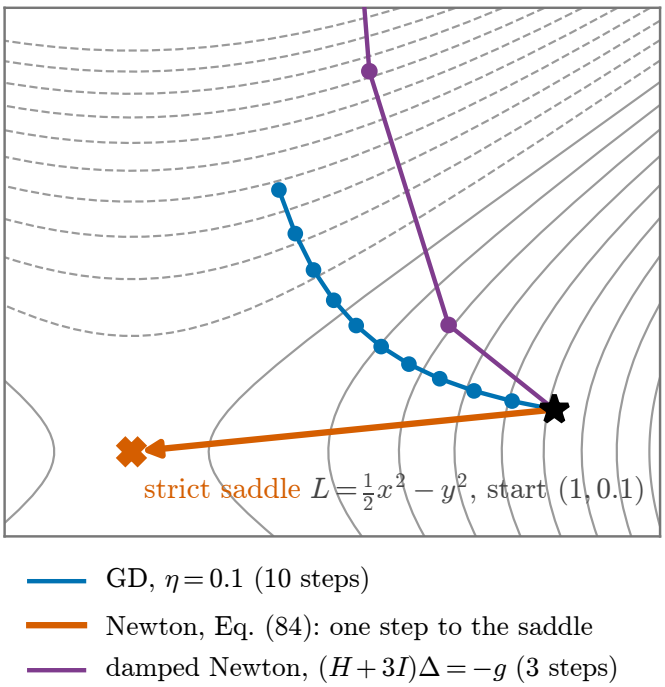


Fig. 14: The saddle of Eq. (82) from $(1, 0.1)$. GD (Eq. (83), $\eta = 0.1$) contracts x and expands y, moving away from the saddle; the unrestricted Newton step of Eq. (84) lands exactly on it. Damping as in Eq. (44) with $\lambda = 3$ makes $H + \lambda I$ positive definite, and the step again escapes along y.

Newton reaches the *saddle* in one step! This is not a contradiction: Newton's equation finds a stationary point of a quadratic approximation. A nonconvex quadratic can have a stationary point that is not a local minimum. The example motivates damping, trust regions, cubic regularization, negative-curvature steps, and second-order stationarity criteria [45], [55], [56]. It also exposes why saying "second order is better because it knows the curvature" is incomplete.

The same phenomenon appears in higher dimensions. If H has both positive and negative eigenvalues, an unconstrained Newton direction $-H^{-1}g$ may point partly uphill, and the quadratic model is unbounded below along negative eigendirections. A trust-region constraint keeps the candidate displacement finite and allows an algorithm to exploit negative curvature when appropriate. But the exact trust-region solution and acceptance logic have computational costs. The correct comparison is not whether a method uses curvature, but whether it uses *the right curvature model and safeguards* for the region it encounters.

B. Stability of Preconditioned Gradient Descent

Suppose locally that the loss is the positive-definite quadratic $L(\theta) = \frac{1}{2}\theta^\top H\theta$, and apply a fixed symmetric positive-definite preconditioner M:

$$\theta_{t+1} = (I - \eta M^{-1}H)\theta_t. \tag{85}$$

TABLE IX: Equation (87) evaluated for three fixed preconditioners on the rotated quadratic of Eq. (72). The multipliers $1 - \eta\lambda(K)$ at $\eta = 1$ explain the one-step results of Appendix C-A and Fig. 11.

Preconditioner M	eigenvalues of K	stable for	multipliers at $\eta = 1$
I (GD)	1, 100	$0 < \eta < 0.02$	0, −99 (unstable)
$\mathrm{diag}(H) = 50.5I$	0.0198, 1.980	$0 < \eta < 1.010$	0.980, −0.980
H (Newton)	1, 1	$0 < \eta < 2$	0, 0

Although $M^{-1}H$ need not be symmetric, it is similar to the symmetric positive-definite matrix

$$K = M^{-1/2}HM^{-1/2}. \tag{86}$$

To verify the similarity, multiply $M^{-1}H$ on the left by $M^{1/2}$ and the right by $M^{-1/2}$; the result is K. Thus $M^{-1}H$ has real positive eigenvalues equal to those of K. The preconditioned error contracts in every eigenmode if

$$\boxed{0 < \eta < \frac{2}{\lambda_{\max}(K)}.} \tag{87}$$

For ordinary GD, $M = I$ and $K = H$, recovering the textbook threshold. For exact Newton preconditioning on a positive-definite quadratic, $M = H$, so $K = I$ and a unit step sends every eigencomponent to zero. For approximate or stale preconditioners, the eigenvalues of K may spread again. This calculation provides a common language for understanding conditioning and the local stability of several variable-metric methods [5], [6].

For changing stochastic preconditioners M_t, however, the analysis is more difficult: M_t may depend on noisy gradients and need not commute with the Hessian; the objective may become non-convex along the trajectory; and the relevant matrices may be singular or indefinite. One must not apply Eq. (87) as if it were an Adam, Shampoo, or K-FAC convergence theorem. It is an exact statement about a local fixed quadratic model with a fixed positive-definite preconditioner.

C. A Short Conceptual Checklist for Reading Optimizer Papers

Before accepting an optimizer's theoretical or empirical claim, identify four objects explicitly. First, the *objective class*: is the loss convex, strongly convex, smooth nonconvex, or a particular network architecture? Second, the *information oracle*: does each update require gradients, Hessian–vector products, factor inverses, extra passes, or a full linear solve? Third, the *success criterion*: training loss, gradient norm, first- or second-order stationarity, validation accuracy, token throughput, or total compute? Fourth, the *operating regime*: minibatch size, precision, model shape, accelerator, damping, and hyperparameter-search budget. These objects determine whether two results are comparable.

For example, a proof of fast local Newton convergence and a report that Muon trains a transformer faster than

AdamW do not conflict: one concerns a regular local minimizer and mathematical iteration error, while the other concerns a concrete stochastic implementation, model architecture, and hardware-dependent cost. Similarly, a convex regret guarantee for Shampoo and a non-convex stationarity result for SGD answer different mathematical questions. The role of a rigorous survey is not to declare a winner from these dissimilar statements but to explain what the statements do and do not establish [9], [68], [74].

References

[1] H. Robbins and S. Monro, "A stochastic approximation method," Annals of Mathematical Statistics, vol. 22, pp. 400–407, 1951. [Online]. Available: https://doi.org/10.1214/aoms/1177729586.

[2] J. Kiefer and J. Wolfowitz, "Stochastic estimation of the maximum of a regression function," Annals of Mathematical Statistics, vol. 23, pp. 462–466, 1952. [Online]. Available: https://doi.org/10.1214/aoms/1177729392.

[3] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," USSR Computational Mathematics and Mathematical Physics, vol. 4, pp. 1–17, 1964. [Online]. Available: https://doi.org/10.1016/0041-5553(64)90137-5.

[4] Y. Nesterov, "A method of solving a convex programming problem with convergence rate $O(1/k^2)$," Soviet Mathematics Doklady, vol. 27, pp. 372–376, 1983.

[5] Y. Nesterov, "Introductory Lectures on Convex Optimization: A Basic Course," Kluwer Academic Publishers, 2004. [Online]. Available: https://doi.org/10.1007/978-1-4419-8853-9.

[6] J. Nocedal and S. Wright, "Numerical Optimization, 2nd ed.," Springer, 2006. [Online]. Available: https://doi.org/10.1007/978-0-387-40065-5.

[7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986. [Online]. Available: https://doi.org/10.1038/323533a0.

[8] L. Bottou, "Large-scale machine learning with stochastic gradient descent," Proceedings of COMPSTAT, pp. 177–186, 2010. [Online]. Available: https://doi.org/10.1007/978-3-7908-2604-3_16.

[9] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," SIAM Review, vol. 60, pp. 223–311, 2018. [Online]. Available: https://doi.org/10.1137/16M1080173.

[10] A. Nemirovski, A. Juditsky, G. Lan, and A. Shapiro, "Robust stochastic approximation approach to stochastic programming," SIAM Journal on Optimization, vol. 19, pp. 1574–1609, 2009. [Online]. Available: https://doi.org/10.1137/070704277.

[11] S. Ghadimi and G. Lan, "Stochastic first- and zeroth-order methods for nonconvex stochastic programming," SIAM Journal on Optimization, vol. 23, pp. 2341–2368, 2013. [Online]. Available: https://doi.org/10.1137/120880811.

[12] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," Proceedings of ICML, PMLR 28, pp. 1139–1147, 2013. [Online]. Available: https://proceedings.mlr.press/v28/sutskever13.html.

[13] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," Journal of Machine Learning Research, vol. 12, pp. 2121–2159, 2011. [Online]. Available: https://jmlr.org/papers/v12/duchi11a.html.

[14] T. Tieleman and G. Hinton, "Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude," Neural Networks for Machine Learning, Coursera lecture, 2012.

[15] M. D. Zeiler, "ADADELTA: An adaptive learning rate method," arXiv:1212.5701, 2012. [Online]. Available: https://arxiv.org/abs/1212.5701.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Proceedings of ICLR, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980.

[17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101.

[18] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond," Proceedings of ICLR, 2018. [Online]. Available: https://openreview.net/forum?id=ryQu7f-RZ.

[19] X. Chen, S. Liu, R. Sun, and M. Hong, "On the convergence of a class of Adam-type algorithms for non-convex optimization," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1808.02941.

[20] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar, "Adaptive methods for nonconvex optimization," Advances in Neural Information Processing Systems, 2018. [Online]. Available: https://arxiv.org/abs/1806.09318.

[21] L. Liu et al., "On the variance of the adaptive learning rate and beyond," Proceedings of ICLR, 2020. [Online]. Available: https://arxiv.org/abs/1908.03265.

[22] J. Zhuang et al., "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," Advances in Neural Information Processing Systems, 2020. [Online]. Available: https://arxiv.org/abs/2010.07468.

[23] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive gradient methods with dynamic bound of learning rate," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1902.09843.

[24] R. Ward, X. Wu, and L. Bottou, "AdaGrad stepsizes: Sharp convergence over nonconvex landscapes," Journal of Machine Learning Research, vol. 21, 2020. [Online]. Available: https://jmlr.org/papers/v21/18-808.html.

[25] A. D'efossez, L. Bottou, F. Bach, and N. Usunier, "A simple convergence proof of Adam and Adagrad," Transactions on Machine Learning Research, 2022. [Online]. Available: https://arxiv.org/abs/2003.02395.

[26] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, "The marginal value of adaptive gradient methods in machine learning," Advances in Neural Information Processing Systems, 2017. [Online]. Available: https://arxiv.org/abs/1705.08292.

[27] N. S. Keskar and R. Socher, "Improving generalization performance by switching from Adam to SGD," arXiv:1712.07628, 2017. [Online]. Available: https://arxiv.org/abs/1712.07628.

[28] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," Proceedings of ICLR, 2017. [Online]. Available: https://arxiv.org/abs/1608.03983.

[29] L. N. Smith, "Cyclical learning rates for training neural networks," Proceedings of WACV, 2017. [Online]. Available: https://arxiv.org/abs/1506.01186.

[30] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," arXiv:1706.02677, 2017. [Online]. Available: https://arxiv.org/abs/1706.02677.

[31] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," Proceedings of ICML, PMLR 80, pp. 4596–4604, 2018. [Online]. Available: https://proceedings.mlr.press/v80/shazeer18a.html.

[32] X. Chen et al., "Symbolic discovery of optimization algorithms," Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2302.06675.

[33] A. Defazio and K. Mishchenko, "Learning-rate-free learning by D-adaptation," Proceedings of ICML, PMLR 202, 2023. [Online]. Available: https://proceedings.mlr.press/v202/defazio23a.html.

[34] A. Defazio et al., "The road less scheduled," Advances in Neural Information Processing Systems, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2405.15682.

[35] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," Proceedings of AISTATS, PMLR 9, 2010. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html.

[36] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," Proceedings of ICCV, 2015. [Online]. Available: https://arxiv.org/abs/1502.01852.

[37] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," Proceedings of ICML, PMLR 37, 2015. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.html.

[38] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016. [Online]. Available: https://arxiv.org/abs/1607.06450.

[39] N. S. Keskar et al., "On large-batch training for deep learning: Generalization gap and sharp minima," Proceedings of ICLR, 2017. [Online]. Available: https://arxiv.org/abs/1609.04836.

[40] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio, "Sharp minima can generalize for deep nets," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/dinh17b.html.

[41] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," Proceedings of ICLR, 2021. [Online]. Available: https://arxiv.org/abs/2010.01412.

[42] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient descent on neural networks typically occurs at the edge of stability," Proceedings of ICLR, 2021. [Online]. Available: https://arxiv.org/abs/2103.00065.

[43] Y. N. Dauphin et al., "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," Advances in Neural Information Processing Systems, 2014. [Online]. Available: https://arxiv.org/abs/1406.2572.

[44] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," Proceedings of COLT, PMLR 49, 2016. [Online]. Available: https://proceedings.mlr.press/v49/lee16.html.

[45] C. Jin et al., "How to escape saddle points efficiently," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/jin17a.html.

[46] R. Ge, F. Huang, C. Jin, and Y. Yuan, "Escaping from saddle points—Online stochastic gradient for tensor decomposition," Proceedings of COLT, PMLR 40, 2015. [Online]. Available: https://proceedings.mlr.press/v40/Ge15.html.

[47] H. Karimi, J. Nutini, and M. Schmidt, "Linear convergence of gradient and proximal-gradient methods under the Polyak–Lojasiewicz condition," Proceedings of ECML PKDD, 2016. [Online]. Available: https://arxiv.org/abs/1608.04636.

[48] C. G. Broyden, "The convergence of a class of double-rank minimization algorithms: 2. The new algorithm," IMA Journal of Applied Mathematics, vol. 6, pp. 222–231, 1970. [Online]. Available: https://doi.org/10.1093/imamat/6.3.222.

[49] R. Fletcher, "A new approach to variable metric algorithms," The Computer Journal, vol. 13, pp. 317–322, 1970. [Online]. Available: https://doi.org/10.1093/comjnl/13.3.317.

[50] D. Goldfarb, "A family of variable-metric methods derived by variational means," Mathematics of Computation, vol. 24, pp. 23–26, 1970. [Online]. Available: https://doi.org/10.1090/S0025-5718-1970-0258249-6.

[51] D. F. Shanno, "Conditioning of quasi-Newton methods for function minimization," Mathematics of Computation, vol. 24, pp. 647–656, 1970. [Online]. Available: https://doi.org/10.1090/S0025-5718-1970-0274029-X.

[52] J. Nocedal, "Updating quasi-Newton matrices with limited storage," Mathematics of Computation, vol. 35, pp. 773–782, 1980. [Online]. Available: https://doi.org/10.1090/S0025-5718-1980-0572855-7.

[53] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," Mathematical Programming, vol. 45, pp. 503–528, 1989. [Online]. Available: https://doi.org/10.1007/BF01589116.

[54] A. R. Conn, N. I. M. Gould, and P. L. Toint, "Trust Region Methods," SIAM, 2000. [Online]. Available: https://doi.org/10.1137/1.9780898719857.

[55] Y. Nesterov and B. T. Polyak, "Cubic regularization of Newton method and its global performance," Mathematical Programming, vol. 108, pp. 177–205, 2006. [Online]. Available: https://doi.org/10.1007/s10107-006-0706-8.

[56] C. Cartis, N. I. M. Gould, and P. L. Toint, "Adaptive cubic regularisation methods for unconstrained optimization. Part I: Motivation, convergence and numerical results," Mathematical Programming, vol. 127, pp. 245–295, 2011. [Online]. Available: https://doi.org/10.1007/s10107-009-0286-5.

[57] B. A. Pearlmutter, "Fast exact multiplication by the Hessian," Neural Computation, vol. 6, pp. 147–160, 1994. [Online]. Available: https://doi.org/10.1162/neco.1994.6.1.147.

[58] N. N. Schraudolph, "Fast curvature matrix-vector products for second-order gradient descent," Neural Computation, vol. 14, pp. 1723–1738, 2002. [Online]. Available: https://doi.org/10.1162/08997660260028683.

[59] J. Martens, "Deep learning via Hessian-free optimization," Proceedings of ICML, 2010. [Online]. Available: https://icml.cc/Conferences/2010/papers/458.pdf.

[60] J. Martens and I. Sutskever, "Learning recurrent neural networks with Hessian-free optimization," Proceedings of ICML, 2011. [Online]. Available: https://www.cs.toronto.edu/~jmartens/docs/HF-RNN.pdf.

[61] S.-I. Amari, "Natural gradient works efficiently in learning," Neural Computation, vol. 10, pp. 251–276, 1998. [Online]. Available: https://doi.org/10.1162/089976698300017746.

[62] R. Pascanu and Y. Bengio, "Revisiting natural gradient for deep networks," Proceedings of ICLR, 2014. [Online]. Available: https://arxiv.org/abs/1301.3584.

[63] J. Martens, "New insights and perspectives on the natural gradient method," Journal of Machine Learning Research, vol. 21, 2020. [Online]. Available: https://jmlr.org/papers/v21/17-678.html.

[64] F. Kunstner, L. Balles, and P. Hennig, "Limitations of the empirical Fisher approximation for natural gradient descent," Advances in Neural Information Processing Systems, 2019. [Online]. Available: https://arxiv.org/abs/1905.12558.

[65] A. Botev, H. Ritter, and D. Barber, "Practical Gauss–Newton optimisation for deep learning," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/botev17a.html.

[66] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," Proceedings of ICML, PMLR 37, 2015. [Online]. Available: https://proceedings.mlr.press/v37/martens15.html.

[67] R. Grosse and J. Martens, "A Kronecker-factored approximate Fisher matrix for convolution layers," Proceedings of ICML, PMLR 48, 2016. [Online]. Available: https://proceedings.mlr.press/v48/grosse16.html.

[68] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," Proceedings of ICML, PMLR 80, 2018. [Online]. Available: https://proceedings.mlr.press/v80/gupta18a.html.

[69] R. Anil et al., "Scalable second order optimization for deep learning," arXiv:2002.09018, 2020. [Online]. Available: https://arxiv.org/abs/2002.09018.

[70] Z. Yao et al., "AdaHessian: An adaptive second order optimizer for machine learning," Proceedings of AAAI, 2021. [Online]. Available: https://arxiv.org/abs/2006.00719.

[71] X. Ma, "Apollo: An adaptive parameter-wise diagonal quasi-Newton method for nonconvex stochastic optimization," Proceedings of AAAI, 2021. [Online]. Available: https://arxiv.org/abs/2009.13586.

[72] H. Liu, Z. Li, D. Hall, P. Liang, and T. Ma, "Sophia: A scalable stochastic second-order optimizer for language model pre-training," Proceedings of ICLR, 2024. [Online]. Available: https://arxiv.org/abs/2305.14342.

[73] N. Vyas et al., "SOAP: Improving and stabilizing Shampoo using Adam," arXiv:2409.11321, 2024. [Online]. Available: https://arxiv.org/abs/2409.11321.

[74] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," Technical article and open-source implementation, 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/.

[75] J. Bernstein and L. Newhouse, "Old optimizer, new norm: An anthology," arXiv:2409.20325, 2024. [Online]. Available: https://arxiv.org/abs/2409.20325.

[76] P. Micikevicius et al., "Mixed precision training," Proceedings of ICLR, 2018. [Online]. Available: https://arxiv.org/abs/1710.03740.

[77] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," Proceedings of SC20, 2020. [Online]. Available: https://arxiv.org/abs/1910.02054.

[78] J. Hoffmann et al., "Training compute-optimal large language models," arXiv:2203.15556, 2022. [Online]. Available: https://arxiv.org/abs/2203.15556.

[79] J. Kaplan et al., "Scaling laws for neural language models," arXiv:2001.08361, 2020. [Online]. Available: https://arxiv.org/abs/2001.08361.

[80] M. R. Zhang et al., "Lookahead optimizer: k steps forward, 1 step back," Advances in Neural Information Processing Systems, 2019. [Online]. Available: https://arxiv.org/abs/1907.08610.